

Detecting Memory Safety Violations in Binary-Only Programs Using Symbolic Execution and Address Sanitisation

Bachelor's Thesis

Faculty of Science, University of Basel
Department of Mathematics and Computer Science
Databases and Information Systems (DBIS) Group
<https://dbis.dmi.unibas.ch/>

Examiner: Dr. Marco Vogt
Supervisor: Prof. Dr. Christopher Scherb

Emanuele Tirendi
emanuele.tirendi@unibas.ch
21-052-089

28.01.2026

Acknowledgements

I want to express my sincere gratitude to Prof. Dr Christopher Scherb for his continuous supervision and guidance throughout the course of this thesis. I am particularly thankful for the trust he placed in me by allowing me to choose a topic closely aligned with my personal interests, which significantly increased my engagement with the research.

I am grateful to Dr Marco Vogt for serving as examiner and for enabling the opportunity to pursue this research topic as well as on his guidance on structuring and presenting academic work.

I would also like to thank Alexandre Warin, my former mathematics teacher at the Gymnasium Münchenstein, who supervised my Matura thesis *Verallgemeinerung der Turingberechenbarkeit mit Betrachtung des Halteproblems*. His openness in allowing me to choose an unconventional, theoretical topic played a decisive role in shaping my academic interests. This early encouragement was a factor in my decision to study Computer Science.

I would like to express my deepest gratitude to my parents for their unwavering support throughout my studies. Their continuous encouragement and support provided me with a stable foundation during both challenging and rewarding times. Their support significantly contributed to the quality of this thesis.

Finally, I would like to express my deepest gratitude to my brother for his emotional support, for being a person of strong character and good values, and for the many intellectually enriching conversations that helped shape my thinking.

Abstract

Symbolic execution is a powerful program analysis technique that enables formal reasoning about execution paths by treating program inputs as symbolic values rather than concrete data. Address sanitisation, in contrast, is a dynamic analysis approach that detects invalid memory accesses, such as buffer overflows, by maintaining a shadow memory at runtime. Binary-only program analysis is particularly relevant in settings where source code is unavailable or cannot be assumed to reflect the executed program faithfully, for instance, due to compiler optimisations.

This thesis investigates the combination of symbolic execution and address sanitisation for detecting invalid memory access vulnerabilities in binary-only programs using the symbolic execution engine ANGR. We first analyse existing symbolic memory models and identify the model that offers the best trade-off between correctness and resource usage for this setting. Based on this foundation, we identify the classes of memory access vulnerabilities detectable by the proposed approach. We then analyse data structures and algorithms for implementing shadow memory with respect to space consumption, runtime overhead, and solver interaction. In addition, we briefly discuss the applicability of established path-explosion mitigation techniques and outline their potential integration as future work. Guided by this analysis, we implement a proof-of-concept for heap memory vulnerability detection within the ANGR framework.

The resulting prototype demonstrates the feasibility of the proposed approach and successfully identifies memory safety violations in binary programs. While the current implementation exhibits notable overhead for programs with highly symbolic behaviour, it provides a solid foundation for further optimisation. In particular, integrating path explosion mitigation techniques discussed in this thesis represents a promising direction for improving scalability and practical applicability.

Contents

1	Introduction	7
2	Theoretical Background	10
2.1	Symbolic Execution	10
2.1.1	How Symbolic Execution Works Conceptually.	10
2.1.2	Soundness and Completeness.	11
2.1.3	Memory Models in Symbolic Execution.	11
2.1.3.1	Memory Models	11
2.1.3.2	Symbolic Memory Accesses	11
2.1.3.3	Space and Time vs Soundness and Completeness	13
2.1.4	Path Explosion Problem and Known Mitigation Techniques.	14
2.2	Address Sanitisation.	15
2.3	Binary-only Analysis.	16
2.3.1	Heap	17
2.3.2	Stack	18
2.3.2.1	Assumptions and Application Binary Interface (ABI)	18
2.3.2.2	Stack Layout	18
2.3.2.3	Dynamic Memory Allocation on the Stack	19
2.3.2.4	Callee-preserved Registers	19
2.3.2.5	Function Calls and Returns	19
2.3.2.6	Non-standard Control Flow: <code>long jmp</code> , Exceptions, and Cloning	19
2.3.2.7	Intentional Wild Dereferences	20
2.3.2.8	Symbolic Instruction Pointer	20
2.3.2.9	Load Widening	20
3	Related Work	21
3.1	S2EDROID	21
3.2	KLEE	23
3.3	UBSYM	23
4	Analysis of Memory Models for Symbolic Execution	25
4.1	Theoretical Analysis of Traditional Models	25
4.2	Analysis of the Data of MEMSIGHT	27
4.3	Conclusion	29
4.4	Unsoundness and Incompleteness Induced by the Memory Model of ANGR	29
4.4.1	Concretisation	29
4.4.2	Hooking of Memory Management Functions	30
4.4.3	Symbolic Write Lengths	30
4.5	Some Proposed Changes	31
4.5.1	Concretisation of Stores	31

4.5.2	Poisoned areas around Heap	32
5	Detection Semantics	33
5.1	Pseudocode Notation	33
5.2	Heap	34
5.2.1	Heap Memory Management	34
5.2.2	Heap Memory Usage	35
5.2.3	Uninitialised Heap Memory	35
5.3	Stack	36
5.3.1	Return Address	36
5.3.2	Red Zone	37
5.3.3	Uninitialised Stack Memory or Use-after-return	38
5.3.4	Longjmp, Exceptions or Cloning	39
5.3.5	Why We Do Not Consider Callee-Preserved Registers and Object Boundaries	40
6	Data Structures and Design	41
6.1	Overview	41
6.1.1	The Concrete Case	41
6.1.2	The Symbolic Case	42
6.2	Shadow Memory Implementations	43
6.2.1	Remarks on Pseudocode and on Notation for Analysis	43
6.2.2	Address-based Shadow Memory	44
6.2.2.1	Concrete-chunk Implementation	44
6.2.2.2	Symbolic-chunk Implementation	45
6.2.3	Range-based Shadow Memory	46
6.2.3.1	Concrete-chunk Implementation	46
6.2.3.2	Symbolic-chunk Implementation	48
6.3	Which one for which Purpose?	48
6.3.1	Heap Memory Management	48
6.3.2	Heap Memory Usage	49
6.3.3	Uninitialised Heap Memory	50
6.3.4	Return Address	51
6.3.5	Uninitialised Stack Memory and Use-after-return	51
7	Compatibility with Path Explosion Mitigation Techniques	52
7.1	Overview	52
7.2	State Merging	53
7.2.1	Concrete-chunk Address-based Shadow Memory	53
7.2.2	Symbolic-chunk Address-based Shadow Memory	54
7.2.3	Range-based Shadow Memory	55
8	Implementation and Evaluation	57
8.1	Implementation	57
8.2	Evaluation	58
9	Discussion	59
10	Conclusion	61

11 Future Work	62
11.1 Memory Models	62
11.2 Detection Semantics	62
11.3 Data Structures and Algorithms for Shadow Memory	62
11.4 Memory Models and Path Explosion	63
11.5 Implementation	63
11.6 Additional Directions	63
12 Usage of AI	64
13 Appendix	65
13.1 Pseudocode for Heap Memory Management Vulnerability Detection Semantics	65
13.2 Pseudocode for Heap Memory Usage Vulnerability Detection Semantics	66
13.3 Pseudocode for Uninitialised Heap Memory Vulnerability Detection Semantics	68

Chapter 1

Introduction

Vulnerabilities originating from invalid memory accesses remain widespread and continue to pose a serious threat to software security. Between 1999 and 2021, more than 13,000 buffer overflow vulnerabilities were reported in the Common Vulnerabilities and Exposures (CVE) database [13]. Beyond buffer overflows, other forms of memory safety violations are similarly prevalent. Out-of-bounds writes, use-after-free errors, and out-of-bounds reads rank among the most dangerous software weaknesses, occupying positions three, seven, and eight, respectively, in the 2025 CWE Top 25 Most Dangerous Software Weaknesses list [5]. A prominent example is the Heartbleed vulnerability disclosed in 2014, which was caused by a buffer overread and led to severe security breaches. According to an eWeek author, the economic damage caused by Heartbleed is estimated at approximately 500 million US dollars [21].

There exist various approaches for detecting invalid memory accesses, which can be categorised along multiple dimensions. These range from manual techniques such as code review to fully automatic methodologies, from approaches based on static analysis to those relying on runtime information (dynamic analysis), and from test-case-based techniques to approaches employing formal reasoning. Among those many approaches, symbolic execution has emerged as particularly powerful. Symbolic execution is a program analysis technique in which programs are executed with symbolic values instead of concrete inputs. During execution, symbolic execution tracks modification on data. When conditional branches are encountered, symbolic execution forks the execution state, exploring all branches while tracking the conditions required to enter each.

Compared to labour-intensive manual techniques such as code review, symbolic execution is fully automatic. In contrast to purely static analysis approaches, which analyse programs without execution, symbolic execution follows actual execution paths and thus incorporates runtime information, allowing for more precise modelling of program behaviour. Furthermore, unlike testing-based methodologies that rely on a finite set of concrete test cases, symbolic execution applies formal reasoning to represent and analyse entire classes of inputs simultaneously. By combining automation, dynamic execution, and formal reasoning, symbolic execution unifies the advantages of these approaches and is therefore a strong candidate for detecting invalid memory accesses.

However, these strengths also come with limitations: symbolic execution suffers from the path explosion problem, which is the exponential growth in the number of feasible execution paths as symbolic execution explores programs with branches, loops, and complex control flow. The path explosion problem significantly restricts the scalability of symbolic execution.

While symbolic execution provides the underlying execution infrastructure for exploring program behaviour, an explicit detection semantics is still required to identify invalid memory accesses; this role can be fulfilled by address sanitisation. Address sanitisation was originally developed for concrete execution rather than symbolic execution and is based on intuitive and practical principles. Address sanitisers maintain a separate representation of program memory, commonly referred to as shadow memory, which stores metadata describing the

validity of memory regions. Through runtime code instrumentation, key events such as memory allocation and deallocation are intercepted, and information is collected, including the size and start of the allocated region. Based on this information, the sanitiser tracks which memory regions are valid or invalid, often referred to as unpoisoned and poisoned memory, respectively. Each memory access is then checked against the shadow memory to determine whether it targets a valid region. Due to its simplicity and effectiveness, address sanitisation has proven to be a powerful technique for detecting memory safety violations in practice.

A natural next step is to combine symbolic execution and address sanitisation in order to leverage the complementary strengths of both approaches. However, this combination introduces substantial challenges in terms of runtime overhead, memory consumption, and the number of solver queries. In particular, symbolic memory accesses, symbolic allocation sizes, and symbolic allocation addresses necessitate a careful choice of an appropriate memory model. Furthermore, the design of efficient data structures and algorithms is crucial to minimise runtime, space usage, and solver interaction incurred by the shadow memory. Finally, maintaining compatibility with existing path-explosion mitigation techniques is essential to prevent scalability issues arising from path explosion.

The applicability of these techniques becomes even more challenging in settings where source code is unavailable. Binary-only analysis is particularly important for several reasons. First, source code is not always available, especially for proprietary or legacy software. Second, even when source code is available, compiler optimisations may significantly alter program structure and behaviour, making only the compiled binary code suitable as ground truth. Consequently, analysing binaries directly is essential for achieving accurate and practical detection of memory safety vulnerabilities in real-world scenarios.

We aim to establish a solid theoretical foundation for combining symbolic execution and address sanitisation in a binary-only setting, thereby enabling future work to build upon our results and develop scalable tools suitable for both academic research and industrial applications. For this, we formulate five objectives for this thesis:

- O_1 : Analyse symbolic memory models and identify the model that offers the best trade-off between correctness and resource usage.
- O_2 : Identify classes of memory access vulnerabilities detectable by the combination of symbolic execution and address sanitisation on binaries.
- O_3 : Design data structures and algorithms for implementing shadow memory with respect to space consumption, runtime overhead, and solver interaction.
- O_4 : Briefly discuss the applicability of established path-explosion mitigation techniques and outline their potential integration as future work.
- O_5 : Implement a proof-of-concept for heap memory vulnerability detection within the ANGR framework [29].

For this thesis, the symbolic execution framework ANGR has been chosen because of its maturity, flexibility, and strong capabilities for analysing binaries without source code. ANGR works directly on executable files and supports multiple instruction sets by lifting assembly code into an intermediate representation. Its modular design gives fine-grained control over symbolic execution, allowing customisation of memory access concretisation, simulation of standard library functions, use of breakpoints for many different types of events, extension of the execution state with so-called plugins, implementation or customisation of state merging, and much more. Additionally, ANGR has an active research community and is an open-source project.

The remainder of this thesis is structured as follows:

- Chapter 2 introduces the theoretical background required for this work.
- Chapter 3 reviews related work combining binary-only analysis, symbolic execution and address sanitisation.
- Chapter 4, analyses symbolic memory models and identifies the model that offers the best trade-off between correctness and resource usage, thus addressing objective O_1 .

- Chapter 5 identifies classes of memory access vulnerabilities detectable by the combination of symbolic execution and address sanitisation on binaries, thereby addressing objective O_2 .
- Chapter 6 designs data structures and algorithms for implementing shadow memory with respect to space consumption, runtime overhead, and solver interaction, by that addressing objective O_3 .
- Chapter 7 discusses the applicability of established path-explosion mitigation techniques and outline their potential integration as future work, thereby addressing objective O_4 .
- Chapter 8 discusses the implementation and evaluation of the developed proof-of-concept, thus addressing objective O_5 .
- The results are discussed in Chapter 9, followed by concluding remarks in Chapter 10 and directions for future work in Chapter 11.

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations necessary for combining symbolic execution with address sanitisation for binary-only analysis and implementing it in ANGR.

2.1 Symbolic Execution

2.1.1 How Symbolic Execution Works Conceptually.

Different symbolic execution engines vary in their design and implementation details; consequently, there is no single, formally standardized definition of symbolic execution tied to an official reference implementation. In this thesis, we therefore present symbolic execution as it is implemented in the framework ANGR. Many modern symbolic execution engines follow conceptually similar designs, as outlined in the comprehensive survey by [9]. Further comparative insights are provided by the work introducing the MEMSIGHT memory model [11], which evaluates and contrasts the memory models of symbolic execution engines ANGR, BINSEC/SE [17], FUZZBALL [23], and KLEE [14]. Additional reference is made to the symbolic execution engine MAYHEM [15], which employs a closely related execution model.

As in concrete execution, symbolic execution maintains a program state consisting of memory and registers and proceeds by executing instructions sequentially. However, when an instruction introduces data whose concrete value is unknown or not fully determined by the execution history - such as user input, nondeterministic behavior, or uninitialised memory - symbolic execution represents this data using symbolic variables. For example, in a statement such as `a = get_user_input()`, concrete execution would assign a specific value (e.g., 3) to the variable `a`, whereas symbolic execution assigns a fresh symbolic variable (e.g., `symb_var_1`) and continues execution while preserving the symbolic nature of `a`.

Symbolic execution tracks operations on symbolic values in the same way concrete execution tracks operations on concrete values. For instance, when executing the statement `a = 2 * a`, concrete execution would update `a` to the value 6, while symbolic execution would update `a` to the symbolic expression `2 * symb_var_1`.

When symbolic execution encounters a conditional branch whose predicate depends on symbolic data, such as `if a > 7 then block_1() else block_2()`, it forks the current program state and explores both branches. Each forked state is augmented with a path constraint that captures the condition under which that execution path is feasible. In this example, the execution entering `block_1()` would record the constraint `symb_var_1 > 3.5`, while the execution entering `block_2()` would record the constraint `symb_var_1 <= 3.5`.

By maintaining symbolic expressions and associated path constraints, symbolic execution enables formal reasoning about program behaviour at any point during execution. This allows properties such as reachability,

correctness conditions, and the presence of runtime errors to be analysed using constraint solvers.

2.1.2 Soundness and Completeness.

We adopt the terminology introduced in [9]. In the context of symbolic-execution-based tools for detecting software bugs, the notions of true and false positives and negatives are commonly used to characterise analysis results. For a given program, a positive result indicates that the tool reports the presence of a bug, whereas a negative result indicates that the tool reports the absence of a bug in that program. A tool that produces false negatives — that is, claims that the program is bug-free even though there is a bug — is considered unsound. Conversely, a tool that produces false positives — that is, reports a bug where none exists — is considered incomplete.

As discussed in [31] and [26], given a formal specification of a software bug, no symbolic execution based tool can simultaneously be sound, complete, and terminating while deciding the presence of that bug for arbitrary programs. Any such tool must necessarily sacrifice at least one of the mentioned properties. These limitations are consistent with Rice’s theorem [27], which asserts that no non-trivial semantic property — such as the presence or absence of a bug — is decidable by an algorithm operating over a Turing-complete language, as is the case for symbolic execution engines. In practice, these limitations manifest in the fact that many programs exhibit a potentially infinite number of execution paths that symbolic execution would need to explore. Attempting to execute all such paths during symbolic execution would result in non-termination. Conversely, selectively pruning execution paths to ensure termination inevitably introduces unsoundness or incompleteness.

2.1.3 Memory Models in Symbolic Execution.

2.1.3.1 Memory Models

A variety of memory models have been proposed for symbolic execution. A detailed comparative discussion is provided in [11], which examines the memory models employed by four symbolic execution engines—ANGR [29], KLEE [14], BINSEC/SE [17], and FUZZBALL [23] — and introduces a fifth model, MEMSIGHT.

According to the analyses presented in [11], as well as the MAYHEM paper [15], symbolic memory states typically follow the same conceptual principle as concrete memory: a mapping from address-like entities to stored values. We deliberately use the term address-like objects rather than addresses, as some symbolic execution engines don’t always use concrete addresses. For instance, MEMSIGHT supports symbolic addresses — addresses that depend on symbolic input — whereas ANGR restricts addresses to be concrete. Consequently, we assume a standard symbolic memory model based on mappings from address-like objects to values and do not further consider alternative representations. The only exception to this is BINSEC/SE which rather uses formulas to describe the memory instead of using the mentioned mappings. But since this approach doesn’t support state merging according to [11], we consider it as being very limited and thus don’t consider it further.

Within such a symbolic memory representation, data whose values are uniquely determined — e.g., because they do not depend on user input, nondeterminism, or uninitialised state — are stored as concrete values. Conversely, data whose values are not uniquely determined are stored as symbolic expressions, such as symbolic variables, as described in Section 2.1.1.

2.1.3.2 Symbolic Memory Accesses

In symbolic execution, memory accesses may target addresses whose values depend on symbolic input. Such addresses are called symbolic addresses. Handling memory operations involving symbolic addresses is a central challenge in symbolic execution, and multiple approaches which we will present here have been proposed.

Consider a memory operation (load or store) targeting a symbolic address i , which may evaluate to any concrete address $i_0, \dots, i_n$. Let the values stored at these addresses be $v_0, \dots, v_n$. Assume a memory model which

employs a mapping from concrete addresses to potentially symbolic values. We now summarise established memory models for handling such memory accesses, following the summary provided by [9].

- **Fully symbolic memory addresses:** These are memory models that can handle symbolic addresses without the need of introducing some sort of restriction on them. There are two different models:
 - *State forking:* For each value $i_j \in i_0, \dots, i_n$, a separate symbolic execution state is created. Each state assumes a distinct address value by adding the constraint $i == i_j$ to its solver and continues execution independently.
 - *ITE expressions:* The execution state is not forked, but ITE (if-then-else) expressions are used. ITE expressions have the form $ite(c, t, f)$ where t and f are values to which the ITE expression might evaluate to, and c is a boolean condition. The ITE expression $ite(c, t, f)$ evaluates to t if condition c holds and to f otherwise. Using ITE expressions, loads and stores are handled as follows:
 - * *Store:* When storing a value v at symbolic address i , the value of each potential address i_j is updated from v_j to an expression of the form $ite(i = i_j, v, v_j)$.
 - * *Load:* A load from symbolic address i yields a nested ITE expression of the form $ite(i = i_0, v_0, ite(i = i_1, v_1, ite(i = i_2, v_2, \dots)))$.
- **Concretised memory addresses:** A concretisation strategy resolves a symbolic address i to a single concrete address i_j , and execution continues under the constraint $i == i_j$.
- **Hybrid approaches:** Symbolic addresses may be partially concretised, resulting in a restricted set of feasible concrete addresses. Depending on the applied strategy, this may lead to state forking or the construction of ITE expressions. Hybrid approaches are employed by default in both ANGR and MAYHEM.
 - ANGR: Store operations are always fully concretised, selecting the maximum feasible address value. For load operations, the minimum and maximum feasible addresses are computed. If their difference exceeds a predefined threshold, the load is fully concretised; otherwise, an ITE expression over the remaining feasible addresses is constructed. ANGR allows these concretisation strategies to be customised by the user.
 - MAYHEM: Follows a similar design. Store operations are always concretised, while load operations are treated analogously to ANGR, but with additional optimisations to reduce solver interaction and execution overhead.

Another, fully symbolic implementation that doesn't fit in the above categorisation is MEMSIGHT. We now briefly describe the base version of its memory model as presented in [11], which is shown in Algorithm 1. This version is not the final iteration of the model, as subsequent refinements introduce soundness-preserving optimisations; nevertheless, it suffices to illustrate the core concepts.

Algorithm 1 MEMSIGHT - base version

<pre> 1: function STORE(e, v) 2: $t \leftarrow t + 1$ 3: $M \leftarrow M \cup \{(e, v, t, true)\}$ 1: function LOAD(e) 2: $v \leftarrow 0$ 3: for $x \in M$ by ascending timestamp do 4: $v \leftarrow ite(e = x.e \wedge x.\delta, x.v, v)$ 5: return v </pre>	<pre> 1: function MERGE($S_{this}, \delta_{this}, S_{other}, \delta_{other}, t_a$) 2: for $\{x \in S_{this}.M x.t > t_a\}$ do 3: $S_{this}.M \leftarrow S_{this}.M _{x.\delta \leftarrow x.\delta \wedge \delta_{this}}$ 4: for $\{x \in S_{other}.M x.t > t_a\}$ do 5: $x.\delta \leftarrow x.\delta \wedge \delta_{other}$ 6: $S_{this}.M \leftarrow S_{this}.M \cup \{x\}$ 7: $S_{this}.t \leftarrow \max(S_{this}.t, S_{other}.t)$ </pre>
---	--

- **Concept:** Symbolic memory M is represented as a collection of tuples (e, v, t, δ) , where e is an expression representing a memory address, v is an expression representing the stored value, t denotes a logical timestamp, and δ is a predicate specifying the path condition under which the tuple is valid.
- **initialisation:** Execution begins with an empty memory set M , a timestamp initialised to 0, and an empty path condition. All memory locations are initially assumed to contain zero.
- $store(e, v)$: Upon storing value v at the potentially symbolic address expression e , the timestamp is incremented and a new tuple (e, v, t, true) is added to M .
- $load(e)$: Load operations construct an ITE expression by comparing the potentially symbolic e against all previously stored address expressions, prioritising the most recent writes. If no match is found, the default value zero is returned.
- **Fork execution:** If execution is forked at a branch with branch condition c then one successor state stores as its predicate δ the value $\delta \wedge c$ while the other one stores as its predicate δ the value $\delta \wedge \neg c$. This does not affect the tuples in M yet.
- $merge(S_{this}, \delta_{this}, S_{other}, \delta_{other}, t_a)$: Here state S_{other} is merged into state S_{this} . The predicates δ_{this} and δ_{other} are the predicates which store the path conditions. The timestamp t_a is the timestamp of the immediate dominator of two branches, which made fork the state into S_{this} and S_{other} . When merging two execution states, all tuples created after timestamp t_a are annotated with their respective path conditions δ_{this} or δ_{other} and then merged into M .

2.1.3.3 Space and Time vs Soundness and Completeness

Memory models that support fully symbolic addresses typically incur substantial computational overhead, leading to excessive runtime or memory consumption during symbolic execution. In contrast, hybrid or fully concretised approaches improve scalability but do so at the cost of soundness and/or completeness. Consequently, the design of practical symbolic execution systems requires an explicit trade-off between correctness (i.e., soundness and completeness) and resource efficiency in terms of time and space.

The paper introducing MAYHEM [15] reports that a hybrid concretisation strategy increases vulnerability detection by approximately 40% compared to full address concretisation. However, the paper does not quantify how many detections are missed relative to a fully symbolic memory model. While it argues that avoiding concretisation altogether would lead to prohibitive runtime overhead, no concrete performance measurements are provided. Instead, it merely notes that systems such as MCNETO [30], BAP [12], and BITBLAZE [3] are capable of reasoning over fully symbolic memory. A closer examination of these works likewise reveals a lack of detailed runtime or scalability evaluations.

The paper introducing MEMSIGHT [11] attempts to analyse this trade-off more systematically by evaluating different memory models on benchmarks authored by the Cromulence (CROMU) DARPA performer group for the Cyber Grand Challenge (CGC) [16]. These benchmarks may be regarded as representative of real-world applications, as realism was an explicit design objective of the CGC benchmark suite.

Specifically, the authors compare three concretisation strategies implemented in ANGR — a fully concretising strategy (CONC), a hybrid strategy (PART), and a fully symbolic strategy (FULL) — as well as the MEMSIGHT memory plugin for ANGR. Based on the characteristics of symbolic memory access patterns, the benchmarks were categorised into three distinct groups:

- **Group 1:** Programs in which symbolic memory indices span a very large range, rendering FULL unable to terminate within the allotted time. The maximum index ranges in these benchmarks vary between 65,536 and 770,048.
- **Group 2:** Programs for which FULL can terminate, but where PART still requires partial concretisation. The maximum index ranges lie between 1 and 8,192.

- **Group 3:** Programs with sufficiently small symbolic memory indices such that PART does not require any concretisation. The maximum index ranges in these benchmarks range from 1 to 28.

Each concretisation strategy was evaluated on every benchmark within each group, and the number of explored execution paths, execution time, and memory consumption were recorded. While the paper reports these measurements, it provides limited aggregation or comparative analysis of the results. As we perform our own aggregation and evaluation of these data in Chapter 4.2, we do not reproduce the raw measurements here. Interested readers are referred to [11] for the original experimental data.

2.1.4 Path Explosion Problem and Known Mitigation Techniques.

As discussed in Section 2.1.2, symbolic execution cannot simultaneously be sound, complete, and guaranteed to terminate. The fundamental reason is the potentially infinite number of execution paths that a program may exhibit. In many contemporary programming languages, such path multiplicity arises from unbounded control-flow constructs, such as `while` loops. However, even in programs without unbounded loops, the number of feasible execution paths typically grows exponentially with the number of conditional branches. While symbolic execution may therefore be sound, complete and terminating in theory for such programs, it does not scale to real-world software systems in practice.

This inherent inability to exhaustively explore all execution paths is known as the *path explosion problem*. In practical settings, exhaustive path exploration is infeasible, necessitating a trade-off between soundness and completeness on the one side, and scalability on the other. Consequently, according to [9] a variety of techniques have been proposed to mitigate the effects of path explosion, including the following:

- **Program or environment modelling:** One approach to mitigating path explosion is to abstract or simplify parts of the program or its execution environment in order to reduce the number of symbolic states. For example, ANGR models many standard library functions using so-called *SimProcedures*. These are Python implementations that directly manipulate the symbolic state, thereby avoiding the overhead of symbolically executing complex library code.
- **Pruning infeasible paths:** Resource usage can be reduced by invoking a constraint solver at each branch point to determine whether a branch is feasible. Although solver invocations introduce computational overhead, early elimination of infeasible paths often leads to more efficient overall execution.
- **Path prioritisation using heuristics:** Heuristic-driven path selection guides symbolic execution toward more promising execution paths, particularly under time or resource constraints. A wide range of heuristics has been proposed, including:
 - **Simple heuristics:** such as depth-first search, breadth-first search, random path selection, and code-coverage-oriented strategies.
 - **Bug-specific heuristics:** as noted in [9], many approaches tailor path prioritisation to detect specific classes of vulnerabilities.
 - **Heuristics for symbolic loop conditions:** when loop bounds depend on symbolic values, heuristics often prioritise executing the loop a limited number of times — typically at least once - while avoiding excessive unrolling, as repeated loop iterations are frequently considered low-yield.
 - **Taint-analysis-driven heuristics:** these approaches prioritise execution paths in which external input (tainted data) influences program behavior.
 - **Program-slicing-driven heuristics:** program slicing identifies instructions relevant to a particular behavior of interest, and paths containing these instructions are prioritized accordingly.
- **Underconstrained symbolic execution:** This technique symbolically executes isolated program fragments without fully modelling their embedding in the complete program. Inputs to such fragments may therefore be underconstrained relative to the global program context. This approach is particularly useful

when the analysis is focused on a specific component or code region.

A central challenge in underconstrained symbolic execution is the handling of complex objects that are provided as inputs to a program fragment. Such objects must already reside in main memory in order to be passed to the fragment. As described in [9], this challenge is commonly addressed through *lazy object evaluation*. In this approach, each field of an input object is initially treated as uninitialised. When a field is accessed during execution, one of the following actions is performed:

1. The field is initialised to an empty or default value. For example, a set is initialised as empty, a memory region is zero-initialised, and a pointer is initialised to `NULL`.
2. If the field is of a simple data type like an integer, those field might just become symbolic.
3. If the field references a complex object, it is replaced by a newly created object whose fields are again uninitialised.
4. If the field references a complex object, it is replaced by a fully concretised instance of this object.

This lazy initialisation strategy allows symbolic execution to reason about complex input objects without eagerly materializing their full structure.

- **Preconditioned symbolic execution:** When analysis is restricted to inputs satisfying specific constraints, these constraints can be explicitly added as preconditions to the symbolic execution. This reduces the search space and focuses the analysis on relevant scenarios.
- **State merging:** Based on the assumption that similar symbolic states lead to similar program behaviour, multiple states may be merged into a single representative state. Depending on the merging strategy, this may preserve all information from the original states or simplify them, potentially introducing unsoundness or incompleteness.

ANGR preserves all information across execution paths, as described in [29] and [11], and thus employs a sound and complete merging strategy. Let `this` and `other` be two states that are to be merged. According to the documentation of ANGR [1] each state is associated with a symbolic Boolean condition, denoted c_{this} and c_{other} , respectively, such that the two conditions are mutually exclusive, while at least one of them must hold.

If, at a given memory address `addr`, state `other` stores the value v_{other} and state `this` stores the value v_{this} , then the merged state stores the conditional expression `ite(c_other, v_other, v_this)` at address `addr`. In this way, state merging preserves the semantics of both execution paths while reducing the number of active states.

But it must be noted that [29] concludes that state merging might also lead to program slowdown since solver query runtime grows exponentially with the number of constraints.

2.2 Address Sanitisation.

Address sanitisation lacks a single formal definition or a canonical implementation; instead, it refers to a class of dynamic analysis techniques for detecting runtime memory safety violations. The concept was popularised by Google’s ADDRESSSANITIZER (ASan) [28], which remains the most widely used and influential implementation. Address sanitisers augment program execution with additional metadata and runtime checks to detect invalid memory accesses, such as out-of-bounds reads and writes, use-after-free errors, and stack buffer overflows.

A central concept in address sanitisation is the use of a separate memory representation, commonly referred to as *shadow memory*. Shadow memory stores metadata that describes the accessibility of regions in the program’s actual memory. Based on this metadata, memory is conceptually divided into *unpoisoned* regions, which

are safe to access, and *poisoned* regions, which must not be accessed. Poisoned regions include, among others, red zones placed around allocated objects, freed memory regions, and unused stack slots.

To maintain this metadata, address sanitiser runtime instrumentation is used. Key events such as memory allocation, deallocation, and stack frame creation are intercepted, allowing the sanitiser to track the base address, size, and lifetime of memory objects. From this information, corresponding regions in the shadow memory are marked as poisoned or unpoisoned. A common implementation strategy maps each byte (or small block) of application memory to a shadow memory entry that encodes whether the corresponding region is accessible. For example, a poisoned byte in normal memory may be represented by zero-valued byte at the same address in shadow memory, while unpoisoned regions are represented by non-zero values, although the exact encoding depends on the implementation. Each memory access is instrumented to consult the shadow memory at runtime; if the access targets a poisoned region, the sanitiser reports an error.

Google’s ADDRESSSANITIZER is enabled as a compile-time option to C and C++ compilers. When applied to source code, this approach is highly effective and efficient, as the compiler has precise knowledge of program structure, data types, and object boundaries. In particular, ASan can accurately identify buffers allocated on the heap and stack, insert poisoned areas around them, and instrument memory accesses accordingly. Inserting poisoned areas around buffers is ultimately done by extending the resulting assembly code such that it allocates buffers far from each other to increase the areas between them which is then marked as poisoned by the shadow memory. These poisoned areas around allocated buffers significantly increase the likelihood of detecting out-of-bounds accesses before they corrupt adjacent objects.

In contrast, applying address sanitisation in a binary-only setting is substantially more challenging and generally less precise for two reasons: First, without source-level information, accurately recovering buffer boundaries — especially for stack-allocated objects — is difficult, if not impossible, without advanced reverse-engineering techniques. And second, binary-only address sanitisers cannot use the source code to alter the resulting assembly code to support address sanitisation by allocating buffers far enough from each other such that the poisoned areas between become larger. As a result, binary-only sanitisers need to rely on limited metadata, for example, poisoning only small regions around heap allocations obtained from intercepted allocation functions. This reduced precision can lead to smaller poisoned regions, allowing some out-of-bounds accesses to reach neighbouring valid objects without being detected, thereby increasing the likelihood of false negatives. On the stack, where object layout and lifetimes are harder to infer from binaries alone, certain classes of memory safety violations may not be detectable at all.

2.3 Binary-only Analysis.

This section discusses the relevant edge cases of heap memory management functions, as well as assumed compiler options.

For stack memory, we explicitly state the assumptions underlying our approach. This is necessary because stack layout and calling conventions may vary across compilers and architectures. We therefore clarify which standards and conventions are assumed. These assumptions implicitly determine which classes of vulnerabilities can be detected and which cannot. In particular, overflows across object boundaries are not detectable, as such boundaries cannot be reliably inferred from assembly-level code alone.

The information obtained in this section will later be used to identify vulnerabilities that can be detected with address sanitisers operating on binaries.

2.3.1 Heap

In this thesis, we restrict our attention to the four standard heap management functions `malloc`, `calloc`, `realloc`, and `free`. For simplicity, we assume that no other functions will allocate or deallocate memory and thus only those four functions can operate on not allocated memory (e.g. for storing chunk metadata). While this assumption may hold for some C programs, it does not generally apply in practice, especially for C++ programs that use `new` and `delete`, or for programs that rely on other standard library, third-party, or custom heap management routines. However, limitation is not inherent to the approach discussed in this thesis. The symbolic execution framework ANGR is highly customisable and allows such functions to be intercepted via hooks or `SIMPROCEDURES`. Consequently, the detection semantics presented in this thesis can be extended to support additional heap management functions with relatively minor implementation effort.

According to the C standard [4], the relative order, alignment, and contiguity of memory regions allocated by successive calls to `malloc`, `calloc`, and `realloc` are unspecified.

In the following we will summarise the semantics of and assumptions for each of the four mentioned heap memory management functions separately.

`ptr = calloc(nmemb, size)`

The C standard specifies the following behavior for `calloc`:

- If the requested memory cannot be allocated, or if the product `nmemb * size` overflows the type `size_t`, `calloc` returns a null pointer.
- If either `nmemb` or `size` is zero, the behavior is implementation-defined: either a null pointer is returned, or the call behaves as if a nonzero size had been requested, except that the returned pointer must not be used to access an object.
- Otherwise, `calloc` returns a pointer to a newly allocated, zero-initialised memory region of sufficient size.

Throughout this thesis, we assume the latter implementation-defined behavior in the zero-size case, i.e., that no null pointer is returned, but the resulting pointer must not be dereferenced.

`free(ptr)`

The C standard defines the behavior of `free` as follows:

- If `ptr` is a valid pointer to the beginning of a previously allocated memory block, the corresponding memory is deallocated.
- If `ptr` is a null pointer, no action is performed.
- If `ptr` is neither a null pointer nor a valid pointer returned by a memory allocation function, the behavior is undefined.

`ptr = malloc(size)`

For `malloc`, the C standard specifies:

- If the requested memory cannot be allocated, `malloc` returns a null pointer.
- If `size` is zero, the behavior is implementation-defined: either a null pointer is returned, or the call behaves as if a nonzero size were requested, except that the returned pointer must not be used to access an object.
- Otherwise, `malloc` returns a pointer to a newly allocated memory region of the requested size.

As with `calloc`, we assume in this thesis that a zero-size allocation does not return a null pointer, but that the returned pointer is non-dereferenceable.

`retptr = realloc(argptr, size)`

The behavior of `realloc` is defined by the C standard through the following cases:

- If `argptr` is neither a null pointer nor a valid pointer to a previously allocated memory block, the behavior is undefined.
- If `argptr` is a null pointer, `realloc` behaves equivalently to `malloc(size)`.
- If `argptr` is valid and non-null and `size` is zero, the behavior is implementation-defined.
- If `argptr` is valid and non-null, `size` is nonzero, and the requested memory cannot be allocated, `realloc` returns a null pointer and leaves the original memory block unchanged.
- If `argptr` is valid and non-null, `size` is nonzero, and allocation succeeds, a pointer to the resized memory block is returned. The contents of the original block are preserved up to the minimum of the old and new sizes.

For the purposes of this thesis, we assume a `realloc` semantics equivalent to the use of the behavior defined by `REALLOC_ZERO_BYTES_FREES` according to [6], whereby calling `realloc` with a valid, non-null pointer and `size == 0` frees the memory block and returns a null pointer.

2.3.2 Stack

2.3.2.1 Assumptions and Application Binary Interface (ABI)

The stack layout is considerably more complex and more dependent on implementation details than heap allocation. Consequently, a precise and well-defined execution model is essential. For this thesis, we assume that all analysed programs conform to the System V Application Binary Interface (ABI), AMD64 Architecture Processor Supplement, Version 1.0 [22].

Unless stated otherwise, all further assumptions made for the stack implementation are explicitly introduced in the following paragraphs under the section 2.3.2.

2.3.2.2 Stack Layout

According to the System V AMD64 ABI [22], the standard calling convention — and thus the corresponding stack layout — is formally defined for global functions. In this work, we assume that local (non-global) functions adhere to the same calling convention.

Position	Contents	Frame
$8n+16(\%rbp)$	memory argument eightbyte n	Previous
...	...	
$16(\%rbp)$	memory argument eightbyte 0	
$8(\%rbp)$	return address	Current
$0(\%rbp)$	previous $\%rbp$ value	
$-8(\%rbp)$	unspecified	
...	...	
$0(\%rsp)$	variable size	
$-128(\%rsp)$	red zone	

Figure 2.1: Stack frame layout according to the System V AMD64 ABI [22]

Figure 2.1 illustrates the stack frame layout as specified by the ABI. The stack grows from higher to lower memory addresses. When a function is invoked, arguments that are passed on the stack are placed in the caller's stack frame. Then the return address is stored on the stack, which becomes the first entry in the callee's stack frame. Subsequently, the previous frame pointer (`rbp`) is saved, followed by the callee's local variables and other frame-specific data.

Below the stack frame lies the so-called red zone, a 128-byte region that may be used by the function. The ABI guarantees that this region will not be clobbered by signal or interrupt handlers.

2.3.2.3 Dynamic Memory Allocation on the Stack

The C and C++ languages permit stack allocations whose size is determined at runtime, for example, through variable-length arrays (VLAs). Consider the declaration `char arr[size]`, where `size` is derived from user input. In such cases, the size of the stack frame becomes input-dependent.

In symbolic execution, this may result in symbolic stack layouts, in which even key elements such as the stack pointer or frame pointer may depend on symbolic values. Depending on the employed memory model, this significantly slows down memory access checks for symbolic address sanitisers because of a much higher number of solver queries.

2.3.2.4 Callee-preserved Registers

The System V AMD64 ABI specifies that the registers `rbp`, `rbx`, and `r12–r15` are callee-preserved [22]. This means that a function must restore its original values before returning to its caller. A callee may either refrain from using these registers or explicitly save and restore their contents.

Empirical inspection of compiler-generated assembly suggests that callee-preserved registers are often saved to the stack when used. However, this behaviour is not mandated by the ABI and is subject to compiler decisions and optimisation levels. As such, we do not assume that these registers are always stored on the stack, and we refrain from relying on this behaviour in the further course of this thesis.

2.3.2.5 Function Calls and Returns

The AMD64 ABI specifies calling conventions and register usage, but does not mandate the use of specific machine instructions for function calls and returns. In principle, implementations may deviate from using the `call` and `ret` instructions while remaining ABI-compliant.

Nevertheless, existing shadow stack systems such as NILE [18], SMASHGUARD [25], and STACKSHIELD [2] consistently assume that function calls are implemented using `call` and returns using `ret`. This is also confirmed by empirical inspection of compiler-generated assembly. Shadow stacks are an IT-security mechanism which, whether implemented in software or hardware, rely on this convention by maintaining a parallel stack of return addresses. Upon function call, they store the return address in the shadow stack, and upon function return, the return address on the regular stack is compared against the shadow stack to detect control-flow hijacking attempts.

Accordingly, throughout this thesis we assume that function calls and returns are implemented using the `call` and `ret` instructions. As discussed in [19], this assumption also holds for the invocation of the `main` function.

2.3.2.6 Non-standard Control Flow: `longjmp`, Exceptions, and Cloning

The function `longjmp`, declared in `<setjmp.h>`, allows non-local control flow transfers. A prior invocation of `setjmp` initialises a buffer of type `jmp_buf` by storing execution context information such as the stack pointer and frame pointer. A subsequent call to `longjmp` restores this saved context.

Such behaviour violates the assumption that every `call` instruction is eventually matched by a corresponding `ret`. The reason for this is that between the call to `setjmp` and `longjmp` many functions could be called without having returned yet. This poses challenges for both shadow stacks and address sanitisers, which typically rely on call–return symmetry to track function entry and exit.

Similar issues arise with C++ exception handling, which transfers control directly from a throwing site to a matching catch block, bypassing intermediate stack frames. Another example is the `clone` system call when

invoked with the `CLONE_VM` and `CLONE_FILES` flags. In this case, the child process shares the parent's address space and stack. If the child executes functions that do not return (e.g., `_exit` or `exec`), shadow stacks and address sanitisers may become inconsistent, leading to spurious errors [28].

2.3.2.7 Intentional Wild Dereferences

As noted in the ADDRESSSANITIZER paper [28], some programs intentionally perform memory accesses that cross stack frame boundaries, for example, by iterating over a contiguous memory region spanning multiple frames. While such accesses may trigger false positives in address sanitisers, they are reported to be rare in practice. We therefore assume that the analysed programs do not contain intentional wild dereferences.

2.3.2.8 Symbolic Instruction Pointer

We assume that the instruction pointer is never symbolic. While symbolic instruction pointers are theoretically possible, we assume that the symbolic executor forks execution whenever the instruction pointer could take on multiple concrete values, thereby maintaining a concrete instruction pointer within each execution state.

2.3.2.9 Load Widening

As discussed in the ADDRESSSANITIZER paper [28], compilers may apply load widening optimisations that combine multiple small memory accesses into a single larger load. Consider the following example:

```
struct X { char a, b, c; };  
void foo() {  
  X x; ...  
  ... = x.a + x.c;  
}
```

Although only individual fields are accessed at the source level, the compiler may emit a single multi-byte load that partially exceeds the object boundary of `x` when constructing the expression `x.a + x.c`. This can result in benign out-of-bounds reads that complicate memory safety checking at the binary level.

Chapter 3

Related Work

In this section, we discuss existing work that addresses problems similar to those considered in this thesis. Among the surveyed approaches, we consider S2EDROID [32] and KLEE [14] to be the most closely related to our work. UBSYM [10] also addresses comparable classes of vulnerabilities, although it follows a different methodology. Other notable approaches that employ symbolic execution to detect related vulnerabilities include MAYHEM [15], MACKE [24], and AEG [8]. However, these systems either operate on source code (AEG), rely on detection mechanisms other than address sanitisation (AEG and MAYHEM), focus primarily on exploit generation rather than vulnerability detection (AEG and MAYHEM), or make use of techniques that are already discussed in this section (e.g., MACKE, which builds upon KLEE).

3.1 S2EDROID

Similar to our approach, S2EDROID [32] employs symbolic execution combined with a shadow memory for binary-only analysis. The tool was originally developed to detect memory corruption vulnerabilities in Android native binaries written in C and C++. S2EDROID is built on selective symbolic execution, a technique that allows parts of a program to be executed symbolically while others are executed concretely.

The core idea of S2EDROID is to define security properties as logical predicates that characterise the absence of memory corruption. In particular, the authors define four rules: two for stack memory and two for heap memory. While we do not reproduce the formal definitions here, the rules can be summarised as follows:

- **Stack rule 1:** No tainted data may access the stored return address or the base pointer.
- **Stack rule 2:** No memory access below the stack pointer is permitted.
- **Heap rule 1:** No tainted data may access the metadata (chunk headers) of allocated or freed heap chunks.
- **Heap rule 2:** No access to freed heap chunks is permitted.

```

Algorithm: Memory Corruption Detection based on
             Selective Symbolic Execution
Input:    path constraint PC; memory operation
             MO; shadow stack SS; shadow heap SH.
Output:   IsVulnerable IV.
Bool MemoryCorruptionDetection() :
    if MO.addr.isSymbolic, then:
        [addr_min, addr_max] = solver
        (MO.addr, PC);
    else:
        addr_min = MO.addr;
        addr_max = MO.addr;
    end if
    size = MO.size + addr_max - addr_min;
    if addr_min is in SS, then:
        IV = StackCheck(SS, Stack.Rule2,
        addr_min);
        if !IV and MO.isWrite, then:
            IV = StackCheck(SS, Stack.Rule1,
            addr_min, size);
        return IV;
    else if addr_min is in SH, then:
        IV = HeapCheck(SH, Heap.Rule2,
        addr_min, size);
        if !IV and MO.isWrite, then:
            IV = HeapCheck(SH, Heap.Rule1,
            addr_min, size);
        return IV;
    end if
    return FALSE;
END

```

Figure 3.1: Pseudocode of S2EDROID

The algorithm used by S2EDROID, illustrated in Figure 3.1, operates as follows. For a memory operation *MO* targeting an address *MO*.addr, the solver is queried to compute the minimum and maximum possible concretisations of the address, denoted *addr_min* and *addr_max*. A size variable *size* is then defined as *MO*.size + *addr_max* - *addr_min*, where *MO*.size represents the size of the memory operation (e.g., the width of a load or store). Based on this information, the following case distinction is applied:

- If the memory operation targets the stack, stack rule 2 is first checked at *addr_min*. If this rule is not violated and the operation is a write, stack rule 1 is checked for the entire interval from *addr_min* to *addr_min* + *size*.
- If the memory operation targets the heap, the corresponding heap rules are checked analogously.

In all cases, a violation of any rule is reported as a vulnerability.

Although S2EDROID follows an approach that is conceptually similar to ours, several important differences exist:

- S2EDROID employs partial symbolic execution, whereas our approach relies on full symbolic execution. While this is a relevant difference in practice, we do not consider it fundamental, as partial symbolic execution primarily serves as an optimisation. Our thesis only considers optimisations by making sure that our approach can be extended with optimisations, but we didn't implement them in our proof-of-concept.
- The rules defined by S2EDROID detect only a subset of the vulnerabilities addressed in this thesis.
- S2EDROID assumes that symbolic addresses correspond to contiguous memory regions and does not consider the possibility that different concretisations of a symbolic address may map to disjoint regions.

While this assumption may hold in many cases, it can lead to false positives by implicitly treating all addresses within the computed minimum and maximum as concretisations of the symbolic address.

- The authors do not specify whether symbolic sizes for heap chunks and stack frames are supported, nor do they provide implementation details of their shadow memory. Based on the available information, we assume that symbolic sizes are not supported and that a relatively simple shadow memory is used; however, this could not be verified since we couldn't find their source code.

3.2 KLEE

Like ANGR [29], KLEE [14] is a symbolic execution engine. Unlike ANGR, however, KLEE provides built-in mechanisms for detecting certain classes of invalid memory accesses, whereas ANGR primarily offers the infrastructure and leaves the detection logic to the user. Furthermore, KLEE operates on the LLVM intermediate representation, while ANGR performs symbolic execution directly on binary code.

As described in the KLEE paper and in the MEMSIGHT paper [11], KLEE does not explicitly model concrete memory layouts such as the heap or stack. Instead, each allocated memory object — whether from the heap or the stack — is conceptually represented as an entry in a map, where the key is an address and the value is the memory object. Object sizes are concretised and cannot be symbolic. When a symbolic memory access is encountered, the solver determines which objects may be affected, and execution is forked for each such object. For each forked state, KLEE checks whether the access is within bounds.

The most important differences between KLEE and our approach are the following:

- KLEE operates on LLVM intermediate code, whereas our approach targets binary code.
- KLEE does not support symbolic object sizes, while our approach allows symbolic sizes by leveraging the memory model of ANGR. The metadata stored in our shadow memory explicitly accounts for symbolic sizes.
- KLEE forks execution when a symbolic memory access may affect multiple objects. In contrast, we handle such cases directly within our shadow memory without forking. The KLEE paper notes that symbolic addresses affecting multiple objects occur relatively infrequently.
- Because LLVM IR does not encode concrete memory layouts, KLEE's memory model abstracts away memory layout information like stack and heap. Our approach explicitly models typical memory layouts, which avoids certain sources of unsoundness or incompleteness introduced by this abstraction.

3.3 UBSYM

UBSYM [10] is a framework designed to detect heap-based buffer overflows, stack-based buffer overflows, use-after-free, and double-free vulnerabilities. Its implementation consists of four main steps:

- Identification of software units that potentially contain buffer overflows using static analysis.
- Symbolic execution of these units in isolation to collect, for each node (corresponding to a basic block), the path constraints required to reach that node. In addition, vulnerability constraints — conditions on the input that trigger the vulnerability — are collected partially based on static analysis as we identified in their source code.
- Application of statistical analysis and learning algorithms to infer a mapping between system-level inputs and the execution of specific units. This is achieved by randomly generating system inputs, executing the program, and observing which units are reached.
- For each vulnerable node executed in the previous step, the collected path and vulnerability constraints are solved to obtain unit-level inputs that trigger the vulnerability. The learned mappings are then used to derive corresponding system-level inputs.

The most notable differences between UBSYM and our approach are the following:

- UBSYM employs several optimisation techniques that we do not use, such as underconstrained symbolic execution and learning-based mappings from system input to unit input. While this is a relevant difference in practice, we do not consider it fundamental. Our thesis only considers optimisations by ensuring that our approach can be extended with them, but we didn't implement them in our proof-of-concept.
- UBSYM relies on static analysis, whereas our approach does not. This constitutes the primary conceptual difference. Although both approaches target similar classes of vulnerabilities and rely on related forms of metadata (e.g., buffer boundaries), the collection and use of metadata differ substantially. In UBSYM, metadata is derived partly from static analysis and partly from symbolic execution. In contrast, our approach collects all metadata during symbolic execution. Vulnerabilities in UBSYM are identified by solving collected vulnerability and path constraints post hoc, whereas in our approach, vulnerabilities are detected directly during symbolic execution, with the corresponding constraints implicitly recorded and implicitly solved when directly extracting the standard input that caused the vulnerability from the ANGR state.

Chapter 4

Analysis of Memory Models for Symbolic Execution

In this chapter, we extend the analysis of existing memory models for symbolic execution introduced in Section 2.1.3.3. Based on this extended analysis, we determine which memory model is most appropriate for the subsequent theoretical investigation and for the implementation of the proof-of-concept presented in this thesis.

Section 4.1 provides a theoretical analysis of traditional memory models by deriving their space requirements for two representative example scenarios. In Section 4.2, we complement this analysis with an evaluation based on previously collected empirical data, enabling a comparative assessment of different models. Section 4.3 synthesises the insights from these analyses to identify the memory models best suited to our objectives. Section 4.4 revisits the sources of unsoundness and incompleteness introduced by the selected memory model and the symbolic execution framework ANGR in more detail. The unsoundness and incompleteness discussed depends on some implementation details of ANGR. The reason why we discuss them in this chapter and not in the chapter targeting our implementation is that those implementation details will influence all the subsequent algorithms, data structures and runtime, space and solver query analyses that we will do.

Finally, in Section 4.5, we discuss potential modifications that could further improve the detection of invalid memory accesses for the chosen memory model.

4.1 Theoretical Analysis of Traditional Models

Let us introduce some unofficial terminology which will simplify the analysis in this section. Let us distinguish between the notions of *logical size* and *representation size* for both symbolic and concrete values. Suppose that, at a given memory location, a symbolic 32-bit integer is stored. Since the value is symbolic, the memory does not contain a concrete integer but rather an expression representing the subset of all 32-bit integers that may reside at that location. Assume that this expression occupies 64 bits. In this case, the logical size is 32 bits, whereas the representation size is 64 bits. An analogous distinction can be made for concrete values, although in practice the logical size and the representation size often coincide.

We further introduce terminology for symbolic addresses. Let i be a symbolic address representing the concrete addresses $i_0, \dots, i_n$, and let each address i_j store a value with representation size r_j . We define the sum $\sum_j r_j$ as the *sum of representation sizes referenced by i* , abbreviated as *SRS* referenced by i , or simply $SRS(i)$.

Having defined this terminology, we now discuss the advantages and disadvantages of the different memory models. We evaluate them by considering two representative scenarios, which we refer to as the best case and the worst case. The rationale for this classification is explained after the analysis since it will be simpler to

understand it then.

- **Case 1 (worst case):** We execute pairs of loads and stores at the same symbolic address i , where i has I concretisations. We perform n such load–store pairs, resulting in a total of $2 \cdot n$ memory operations.
- **Case 2 (best case):** We execute loads and stores n times (again totaling $2 \cdot n$ operations), with each operation accessing a different symbolic address. The j -th store is performed at address s_j and the corresponding load at l_j . We further assume that the sets of concretisations of these symbolic addresses are disjoint and that each symbolic address has exactly I concretisations.

We additionally assume that, initially, in both cases, each concrete address stores a value with representation size 1. Consequently, each symbolic address initially references an SRS of I .

We now compare the behavior of the different memory models.

- **Fully symbolic memory with state forking:** In both Case 1 and Case 2, this model introduces $2 \cdot n$ forking points, each of which forks into I distinct states. As a result, the number of symbolic memory states is multiplied by $I^{2 \cdot n}$.
- **Fully symbolic memory with ITE expressions:** In Case 1, the first load creates an expression whose size is linear in the $SRS(i)$, namely I , since the ITE expression must account for all I concretisations of i , each with representation size 1. The subsequent store writes this expression to I concrete locations, without significantly increasing the expression size. As a result, $SRS(i)$ grows from I to I^2 . The second load then produces an expression of size I^2 , which is again stored at I locations, increasing $SRS(i)$ to I^3 . Repeating this process for all n load–store pairs causes $SRS(i)$ to grow from I to I^{n+1} .
In Case 2, each load–store pair independently increases $SRS(s_j)$ from I to I^2 , leading the total $\sum_j SRS(s_j)$ to grow from $n \cdot I$ to $n \cdot I^2$.
- **Fully concretised memory:** Since all values are concretised, each load returns a value with representation size 1, and each store writes to a single concrete location. Consequently, the overall representation size remains unchanged in both cases.
- **Hybrid approaches:** Consider a hybrid approach in which stores are concretised but loads remain symbolic. In Case 1, the first load produces an expression of size linear in I , while the subsequent store writes this expression to a single concrete location. Thus, $SRS(i)$ increases from I to approximately $2 \cdot I$. The subsequent load produces an expression of size linear to $2 \cdot I$, and the store operation replaces the previously stored one. The $SRS(i)$ increases from $2 \cdot I$ to approximately $3 \cdot I$. After n load–store pairs, $SRS(i)$ grows to approximately $(n + 1) \cdot I$.
In Case 2, each load–store pair increases $SRS(s_j)$ from I to approximately $2 \cdot I$, causing $\sum_j SRS(s_j)$ to increase from $n \cdot I$ to $2 \cdot n \cdot I$.

We classify the two scenarios as the best and worst cases for the following reasons. For the fully symbolic memory with state forking, concretisation always results in state splitting, making it impossible to meaningfully distinguish between best and worst cases. Similarly, the fully concretised model exhibits no variation between best and worst cases. In contrast, the fully symbolic memory with ITE expressions, and to some extent the hybrid model, do exhibit distinct best- and worst-case behaviors. In particular, the fully symbolic model with ITE expressions may incur significant growth in representation size when ITE expressions are repeatedly nested due to reloading already stored ITE expressions. The best case occurs when constructed ITE expressions are not reused in subsequent ITE constructions. The two scenarios were chosen to precisely capture these extremes.

Since the fully symbolic memory with state forking exhibits by far the worst time and memory behaviour due to excessive state forking, we do not consider this model further.

For the remaining models, we summarise the results as follows:

- **Fully symbolic memory with ITE expressions:**
 - Case 1: $SRS(i)$ increases from I to I^{n+1} .

- Case 2: $\sum_j SRS(s_j)$ increases from $n \cdot I$ to $n \cdot I^2$.
- **Fully concretised memory:** All representation sizes remain constant.
- **Hybrid approach:**
 - Case 1: $SRS(i)$ increases from I to $(n + 1) \cdot I$.
 - Case 2: $\sum_j SRS(s_j)$ increases from $n \cdot I$ to $2 \cdot n \cdot I$.

From these results, we observe that the fully symbolic memory with ITE expressions exhibits unacceptable behavior in the worst case and only marginally acceptable behavior in the best case. The fully concretised model trivially achieves the best performance, while the hybrid approach demonstrates acceptable and predictable behavior.

4.2 Analysis of the Data of MEMSIGHT

As discussed in 2.1.3.3, the authors of MEMSIGHT [11] evaluated the three memory models available in ANGR: the fully concretised model CONC, the hybrid model PART, and the fully symbolic model FULL together with the MEMSIGHT plugin for ANGR. Each model was evaluated on three distinct groups of benchmarks:

- **Group 1:** Programs in which symbolic memory indices span very large ranges, causing FULL to fail to terminate within the allotted time.
- **Group 2:** Programs for which FULL does terminate, but where PART still requires partial concretisation.
- **Group 3:** Programs with sufficiently small symbolic memory indices such that PART does not require any concretisation.

For each run, the authors recorded the number of explored execution paths, execution time, and memory consumption, without performing further analysis on these measurements.

We now use this data to analyse which memory model achieves the most favourable trade-off between resource consumption and correctness, where correctness is understood in terms of soundness and completeness. To this end, we define a performance metric of the form

$$performance = \frac{correctness}{resources}.$$

For simplicity, we approximate correctness by the number of execution paths explored by a model. Resource consumption is measured in terms of execution time and memory usage, yielding the two metrics

$$performance_{time} = \frac{\# paths}{time} \quad \text{and} \quad performance_{space} = \frac{\# paths}{memory}.$$

Since our goal is to compare models rather than to assess each model in isolation, all performance values are normalised by dividing them by the corresponding performance of CONC.

It should be noted that this metric inherently penalises models that prioritise correctness, in particular fully symbolic memory models. The distinction regarding correctness between models that concretise and those that do not is not limited to the number of explored paths. In fully symbolic models, each path is typically explored with a higher degree of symbolic information, which can result in executions that are more precise than those obtained under partial or full concretisation. However, as MEMSIGHT does not provide data on the number of constraints or symbolic variables involved in each execution, this aspect cannot be incorporated into our formulas and is therefore disregarded. The aggregated data used in this evaluation is presented in Table 4.1.

Table 4.1: Normalised runtime and memory usage for different memory models

Benchmark	Runtime				Memory			
	CONC	PART	FULL	MEMSIGHT	CONC	PART	FULL	MEMSIGHT
Group 1								
CROMU_00001	1.0	1.91	< 0.03	0.35	1.0	1.72	—	0.79
CROMU_00018	1.0	1.23	<< 0.2	0.24	1.0	1.23	—	0.4
CROMU_00038	1.0	1.11	<< 1.11	1.17	1.0	0.91	—	0.59
Group 2								
CROMU_00001	1.0	1.0	1.11	0.95	1.0	1.0	1.03	1.03
CROMU_00009	1.0	0.91	0.89	0.8	1.0	1.0	0.61	0.58
CROMU_00014	1.0	1.0	0.93	0.86	1.0	1.0	0.86	0.51
CROMU_00024	1.0	1.0	0.36	0.27	1.0	1.0	0.91	0.71
CROMU_00031	1.0	1.31	1.35	1.1	1.0	0.83	0.81	0.8
CROMU_00032	1.0	1.0	0.15	1.52	1.0	1.0	0.62	0.81
CROMU_00033	1.0	1.0	0.76	0.82	1.0	1.0	0.96	0.88
Group 3								
CROMU_00002	1.0	0.91	1.0	0.91	1.0	0.83	1.0	1.0
CROMU_00003	1.0	4.33	4.16	3.9	1.0	2.05	2.19	2.01
CROMU_00004	1.0	1.11	1.0	1.0	1.0	1.0	1.0	0.62
CROMU_00005	1.0	1.0	1.0	0.91	1.0	1.0	1.0	1.0
CROMU_00008	1.0	1.0	1.0	0.91	1.0	1.0	1.0	0.67
CROMU_00012	1.0	0.96	0.88	0.71	1.0	1.18	1.06	1.33
CROMU_00022	1.0	0.91	0.91	0.84	1.0	1.09	0.99	1.21
CROMU_00029	1.0	1.04	0.97	0.92	1.0	1.18	1.38	1.27
CROMU_00040	1.0	0.99	0.99	0.95	1.0	2.18	2.18	1.82
CROMU_00041	1.0	1.0	1.0	0.91	1.0	1.43	1.0	1.43

On the leftmost column, the table lists the benchmarks on which the different memory models were evaluated. All remaining columns report, for each benchmark, the values obtained using the performance formula introduced above, normalized with respect to the corresponding value of CONC. Here the columns under *Runtime* report the values obtained with the formula for $performance_{time}$ while the columns under *Memory* report the values obtained with the formula for $performance_{space}$.

Since FULL did not terminate within the allotted time frame of two hours for the first three benchmarks, no memory performance data is available for these cases. For the time performance, we conservatively assumed that FULL terminated exactly at the two-hour limit and that it explored all execution paths discovered by MEMSIGHT. Under these assumptions, the reported values would represent the performance presented for FULL. However, because the first assumption is known to be false and the validity of the second assumption is uncertain, we explicitly annotate these results as likely underestimating the true performance costs by using the << notation.

As shown in the table, for benchmarks in Group 3 there are no substantial performance differences between PART, FULL, and MEMSIGHT. In Group 2, only the benchmarks CHROMU_00024 for MEMSIGHT and CHROMU_00032 for FULL show noticeably poor runtime performance compared to PART. The most concerning results appear in Group 1, where PART performs at least 120 times better than FULL for CROMU_00001. Furthermore, for CROMU_00001 and CROMU_00018, the performance of MEMSIGHT is approximately five to six times worse than that of PART. So, following the mentioned formulas and the used data, PART shows the best performance of all four candidates.

Another research question addressed by [11] concerns the number of symbolic memory indices in real-world

programs and the typical width of such indices. This aspect is important, since the fewer symbolic memory indices exist, and the narrower their concretisation ranges are, the less significant the tradeoff between performance and correctness becomes. The authors observed that the majority of the benchmarks fall into Group 3.

4.3 Conclusion

From theoretical analysis, we conclude that a fully symbolic memory model with state forking is impractical due to absurd state forking in the worst case, and we therefore do not consider it a viable option.

A fully symbolic memory model based on ITE expressions can, in theory, lead to unacceptable behaviour. However, as demonstrated by the CROMU benchmark data collected by [11] and aggregated in our analysis, the disadvantages primarily show when symbolic memory indices span very wide ranges, and they don't show in other cases. We did not do a theoretical performance analysis for MEMSIGHT due to the complexity of MEMSIGHT's implementation, but the CROMU benchmark data shows that MEMSIGHT's behavior is much more stable than the behavior of FULL. For this reason, if a fully symbolic model were to be used, we would MEMSIGHT over FULL.

The CROMU benchmark results show little difference between PART and MEMSIGHT for benchmarks in Groups 2 and 3, whereas PART performs noticeably better on Group 1. Nevertheless, the employed performance metric remains biased toward models that rely on concretisation. And the benchmark data suggest that symbolic memory indices with wide ranges occur seldom in practice. While the available dataset is highly informative, a substantially larger benchmark suite would be required to draw statistically representative conclusions. Overall, we conclude that both PART and MEMSIGHT exhibit favourable trade-offs between correctness and performance.

We choose PART for our implementation, as it is more widely used and tested than MEMSIGHT, because ANGR SIMPROCEDURES are specifically designed to support its memory model, and because adopting MEMSIGHT would require additional effort to understand its implementation and to ensure compatibility with the ANGR components used in our work.

4.4 Unsoundness and Incompleteness Induced by the Memory Model of ANGR

Since ANGR offers extensive configurability, the resulting analysis properties depend on how the framework is instantiated. For this reason, we discuss unsoundnesses and incompletenesses not of ANGR in general, but of the specific configuration and usage adopted in this thesis, like the usage of ANGR's default memory model.

4.4.1 Concretisation

As already discussed in detail, the default ANGR memory model partially concretises symbolic memory accesses. In particular, store operations to symbolic addresses are concretised to the maximum possible address value. The rationale behind concretising writes to the maximum possible address instead of concretising them to any other value is to support the detection of out-of-bounds accesses, under the common assumption that symbolic memory accesses frequently take the form `arr[idx]`, where `arr` is a concrete base pointer and `idx` is a symbolic index. In such cases, choosing the maximum index value tends to expose potential buffer overflows. However, this strategy also introduces disadvantages, particularly for stack-based allocations.

Consider the following example. Let the stack pointer `rsp` initially hold a concrete value `c`. A variable-length

stack allocation `char arr[size];`, where `size` is symbolic, is typically compiled into an instruction subtracting `size` from `rsp`, yielding the symbolic value $c - \text{size}$. If a subsequent local variable, such as `int a;`, is allocated, `rsp` is further decremented by 4, resulting in the address $c - \text{size} - 4$. When this variable is accessed (e.g., via `a = 3;`), ANGR’s concretisation strategy concretises the symbolic address $c - \text{size} - 4$ by choosing the maximum possible address, which corresponds to setting `size` to zero.

This behavior effectively minimizes the size of the variable-length buffer, whereas from a vulnerability-detection perspective it would often be preferable to maximize it, as using larger buffers as source for copies more likely enable overflows into the destination buffers.

4.4.2 Hooking of Memory Management Functions

The standard heap management functions `malloc`, `calloc`, `realloc`, and `free` are implemented in ANGR as `SIMPROCEDURES`. That is, they are executed as Python code rather than being symbolically executed at the instruction level. While this significantly improves performance, it also affects the analysis precision.

Within these `SIMPROCEDURES`, allocation sizes that are symbolic are maximised when reserving memory. At first glance, this behaviour may appear unsound; however, it is not. ANGR merely reserves address space corresponding to the maximum feasible allocation size, without imposing any constraint that forces the symbolic size to take that value. The symbolic size expression remains unconstrained, and the semantic validity of memory accesses continues to depend on its eventual concretisation. As a result, if the symbolic size is later constrained to a smaller value (e.g., due to a conditional branch), accesses beyond the concretised size remain semantically invalid.

It should be noted that when the maximum feasible allocation size exceeds 128 bytes, ANGR truncates it to 128. This limitation can be circumvented using ANGR’s hooking mechanism, which allows interception of the allocation call, explicit maximisation of the symbolic size, and subsequent invocation of the allocation routine with a concrete value.

Concretisation of allocation sizes influences the placement of heap objects, as real-world memory allocators often choose buffer locations based on the sizes of available free chunks. Consequently, ANGR introduces unsoundness and incompleteness with respect to specific allocator implementations. This does not, however, affect conformance to the C standard, which leaves the placement of allocated objects unspecified. It should be noted that concretisation of allocation sizes leads all the returned pointers by `malloc`, `calloc` and `realloc` to be concrete.

Actual sources of unsoundness and incompleteness arise when pointers passed to `realloc` and `free` are concretised to their maximum possible values.

4.4.3 Symbolic Write Lengths

In principle, writing a data chunk of symbolic length requires an explicit loop whose termination condition depends on symbolic data. Since ANGR forks execution whenever such a condition is encountered, allowing writes of symbolically sized data would result in state forking for each possible length, leading each state to handle a data chunk with concrete value. So, ANGR won’t perform writes of data chunks with symbolic length.

Nevertheless, ANGR provides numerous `SIMPROCEDURES` for standard library functions that could, in theory, model writes of symbolically sized data. Such writes could be implemented using conditional (`ite`) expressions. For example, consider a data chunk `char chunk[size]` with symbolic size `size`, where `chunk[i]` equals `c` for all $i \in 0, \dots, \text{size}$. If this chunk is copied into a memory region initially containing only the value `d` at every location, the implementation could, for each address `addr` between `min(size)` and `max(size)`, store the value `ite(size > addr, c, d)`.

However, based on our investigation — including code review, documentation analysis, and experimental evaluation — we found no evidence that ANGR employs this strategy in practice. Instead, ANGR appears to consistently concretise size arguments in such cases. We note, however, that this conclusion cannot be stated with absolute certainty as our research was not exhaustive. But for the remainder of this thesis, we still assume that ANGR concretises symbolic sizes in writes.

4.5 Some Proposed Changes

We propose changes to the memory model of ANGR. We don't implement them in our proof-of-concept, but summarise them for potential future work.

4.5.1 Concretisation of Stores

A potential improvement we propose is a model similar to PART, in which store operations are not only concretised to the maximum possible value, but to both the maximum and the minimum possible values. The advantage of this approach becomes apparent in the following scenario. Consider two buffers, where b_s denotes the source buffer and b_d the destination buffer, and assume that the entire content of the source buffer is copied into the destination buffer. Let the respective buffer sizes be $size_s$ and $size_d$. When data is written to these buffers for the first time and stores are maximised, the buffers receive sizes $\max(size_s)$ and $\max(size_d)$. Suppose that $\max(size_d) > \max(size_s)$ but $\min(size_d) < \min(size_s)$. Under a purely maximising strategy, no vulnerability would be detected, since the copy operation would not overflow the destination buffer. However, if both sizes were minimised, or if $size_s$ were minimised while $size_d$ were maximised, a buffer overflow could be exposed. So, concretising to both the maximum and minimum could be very helpful.

If the allocation size $size$ of a buffer with base address $base$ is concretised to its minimum and maximum values, the content between $base + \min(size)$ and $base + \max(size)$ as well as the location of all adjacent data depends on the size of the buffer. To give an example, assume that the instruction $base[size-1]=v$ is executed. Then one would need an ITE expression at $base[\min(size)]$ of the form $ite(size = \min(size), v, \dots)$ and one at $base[\max(size)]$ of the form $ite(size = \max(size), v, \dots)$. On the heap, as discussed earlier, ANGR SIMPROCEDURES reserve the maximum possible space for a buffer. Consequently, even when the buffer size is concretised to extreme values, the entire buffer remains within the reserved memory region and does not affect the placement of subsequently allocated data. Figuratively speaking, stretching a heap buffer from its minimum to its maximum size does not shift allocated memory that follows it. This means that maximising and minimising the size of a buffer at the same time only requires the use of ITE expressions between $base + \min(size)$ and $base + \max(size)$, but does not affect adjacent data. So, it can be argued that the use of ITE expressions, as long as it remains between $base + \min(size)$ and $base + \max(size)$, might be an acceptable tradeoff.

This approach, however, introduces complications for stack-based allocations. Dynamic stack allocations do not cause ANGR to reserve the maximum possible space. As a result, concretising a stack buffer not only affects the data within that buffer, but also all the data written next to this buffer. Dealing with this issue would require extensive use of ITE expressions within the buffer and across all data adjacent to it, or even state forking, both of which are undesirable.

An alternative solution would be to implement a dedicated stack plugin that detects such cases and reserves the maximum possible size for stack buffers in the same way as the heap does. But this approach could bring many complications, since the assembly code produced by compilers is not designed to support this. This approach might require code interception and potentially require different implementations for different compilers, and still not recognise all the cases.

Such an implementation would need thorough validation to ensure that program semantics are preserved. But once implemented, such a solution could significantly improve vulnerability detection and would integrate naturally with address sanitisation in binary-only analysis.

4.5.2 Poisoned areas around Heap

Following the approach of allocating stack buffers at least 32-bytes apart from each other as done by the Google ADDRESSSANITIZER, one could modify ANGR's SIMPROCEDURES to do the same for the heap buffers. This would enable detection of some out-of-bounds accesses that remain undetected as of now since they access neighboring buffers which are unpoisoned.

Chapter 5

Detection Semantics

The primary contribution of this chapter is a formalised detection semantics for invalid memory accesses in a binary-only symbolic execution setting. This contribution is presented through abstract algorithms and pseudocode, which serve as an executable specification that is independent of any concrete implementation.

Note that the following algorithms presented assume behavior presented in Section 4.4. Those introduce some sort of unsoundness and incompleteness, but simplify the following algorithms.

Where the pseudocode would become excessively long due to implementation-specific details (e.g. the numerous edge cases of memory management functions), we present only a more abstract, high-level version in the main text. The detailed pseudocode addressing these edge cases is deferred to the appendix.

Section 5.1 introduces the notation used throughout the chapter for the pseudocode. Subsequently, the subsections of Section 5.2 present the algorithms for detecting heap-related vulnerabilities, while the subsections of Section 5.3 describe the algorithms for detecting vulnerabilities on the stack.

5.1 Pseudocode Notation

We first introduce the functions and objects used in the pseudocode presented in the following chapters, which formalize the detection semantics.

- **Solver functions:** The functions `satisfiable()`, `min()`, and `max()` invoke the underlying constraint solver to check satisfiability or compute bounds of symbolic expressions.
- **Program’s memory:** The object `memory` models the memory of the executing program. It exposes the attributes `heap_begin`, `heap_end` and `stack_bound`, which denote the boundaries of the heap and the end of the stack, as well as `max_addr`, representing the maximum address that may exist in the modeled address space. The object further provides the functions for invoking standard library functions (e.g. `memory.malloc()`) or for executing assembly instructions (e.g. `memory.call()`).
- **Shadow memory:** The object `shadow` represents the shadow memory used to track memory validity metadata. It provides the functions `shadow.poison()`, which marks a memory region or an address as invalid and `shadow.unpoison()`, which marks a memory region or an address as valid. The predicate functions `shadow.is_poisoned()` and `shadow.is_unpoisoned()` take an address as input and return a (possibly symbolic) Boolean indicating whether the address is poisoned or unpoisoned, respectively. The functions `shadow.contains_poisoned()` and `shadow.contains_unpoisoned()` take a memory region as input and check whether there is one address in this region which is poisoned or unpoisoned. The return value is again a potentially symbolic Boolean value. Memory regions

spanning from address `addr1` to address `addr2` are denoted by the interval `addr1:addr2`. All addresses, region boundaries, and Boolean results may be symbolic. Consequently, the Boolean results of `shadow.is_poisoned()` and `shadow.is_unpoisoned()` must not be used directly in branch conditions; instead, they should be queried via `satisfiable()`.

- **Metadata:** The object `meta` is a dictionary used to store auxiliary metadata in the form of arbitrary key-value pairs. The object `ret_stack` is a stack storing locations of return addresses.
- **Vulnerability handling:** The function `boom()` represents the detection of a vulnerability and can be interpreted as raising an exception or signaling an error state.

5.2 Heap

5.2.1 Heap Memory Management

This section presents the pseudocode for detecting heap memory management violations, shown in Algorithm 2 as high-level version, and in 12 in more detail. In particular, it targets invalid invocations of heap management routines, such as calls to `free` or `realloc` with pointers that do not reference currently allocated heap objects. Using this instrumentation, we are able to detect several classes of heap-related memory corruption vulnerabilities, including invalid frees, invalid reallocations, double-free errors, and double-realloc errors.

The pseudocode implements an algorithm that marks the beginning of each allocated heap block as unpoisoned. Note that not the whole allocated block is marked as unpoisoned, but only the address pointing to the beginning of the block. Upon every deallocation, it checks whether the pointer argument refers to an unpoisoned (i.e., valid) region or to a poisoned (i.e., invalid) one.

Algorithm 2 Detecting heap memory management violations.

```

1: function INIT( )
2:   shadow.poison(1, memory.max_addr)
3:
4: function ALLOC_INSTRUM(size)
5:   ptr = memory.alloc(size)
6:   shadow.unpoison(ptr)
7:
8: function DEALLOC_INSTRUM(ptr)
9:   if satisfiable(shadow.is_poisoned(ptr)) then
10:    boom()
11:   memory.dealloc(max(ptr))
12:   shadow.poison(max(ptr))

```

The following clarifications apply to the pseudocode:

- The function `init()` performs the initial setup of the shadow memory. All other functions correspond to runtime instrumentation code that intercepts calls to heap memory management routines. All inputs to these functions may be symbolic.
- During instrumentation, the corresponding memory management routines are invoked explicitly (e.g., via `memory.alloc()`). In addition, the shadow memory is updated accordingly (e.g., via `shadow.poison()`). Note that since we assume the behavior introduced in Section 4.4, the pointers returned by the memory management functions like `memory.alloc()` are concrete rather than symbolic.
- Note that we used the functions `memory.alloc()` and `memory.dealloc()` instead of the concrete memory management functions. The pseudocode containing the concrete functions like `memory.malloc()` is presented in 12 as mentioned before.

5.2.2 Heap Memory Usage

This section presents the pseudocode for detecting heap memory usage violations, shown in Algorithm 3 as high-level version, and in 13 in more detail. Specifically, it targets memory accesses to regions of the heap that are not currently allocated. Using this approach, we can detect heap buffer out-of-bounds reads and writes, including the special cases of heap buffer overflows and overreads, accesses to zero-sized buffers, as well as some use-after-free and use-after-realloc vulnerabilities.

Conceptually, the algorithm marks each allocated heap buffer as unpoisoned and checks every memory access to determine whether it targets a poisoned or unpoisoned region.

Algorithm 3 Detecting heap memory usage violations.

```
1: function INIT( )
2:   shadow.unpoison(1, memory.heap_begin - 1)
3:   shadow.poison(memory.heap_begin, memory.heap_end)
4:   shadow.unpoison(memory.heap_end + 1, memory.max_addr)
5:
6: function ALLOC_INSTRUM(size)
7:   ptr = memory.alloc(size)
8:   shadow.unpoison(ptr:ptr+size)
9:   meta[ptr] = size
10:
11: function DEALLOC_INSTRUM(ptr)
12:   memory.dealloc(ptr)
13:   size = meta[ptr]
14:   shadow.poison(ptr:ptr+size)
15:
16: function MEM_ACCESS_INSTRUM(addr, length)
17:   if satisfiable(shadow.contains_poisoned(addr:addr+length)) then
18:     boom()
19:   memory.mem_access(addr)
```

We do not repeat the clarifications already discussed in Section 5.2.1, but instead highlight the following additional aspects:

- During initialisation, all memory regions outside the heap are marked as unpoisoned, as accesses to these regions are not considered invalid heap memory accesses.
- Since heap memory management routines do not explicitly store buffer sizes, we use the `meta` data structure to associate each allocated buffer with its corresponding size. This information is required during deallocation to correctly update the shadow memory.
- The function `mem_access_instrum()` instruments every memory access, whether a load or a store, excluding accesses performed internally by memory management routines. These routines are the only ones permitted to access unallocated heap memory. The instrumentation checks the validity of each access.
- It should be noted that we don't check whether the memory management functions take valid pointers since this is already done by another algorithm.

5.2.3 Uninitialised Heap Memory

This section presents the pseudocode for detecting reads of uninitialised heap memory, shown in Algorithm 4 as high level version, and in 14 in more detailed. The algorithm monitors heap memory reads to determine

whether the accessed memory has previously been initialised (i.e., written to), thereby identifying uninitialised-read violations.

Algorithm 4 Detecting uninitialised heap memory reads.

```
1: function INIT( )
2:   shadow.unpoison(1, memory.heap_begin - 1)
3:   shadow.poison(memory.heap_begin, memory.heap_end)
4:   shadow.unpoison(memory.heap_end + 1, memory.max_addr)
5:
6: function ALLOC_INSTRUM(size)
7:   ptr = memory.alloc(size)
8:   shadow.poison(ptr:ptr+size)
9:   meta[ptr] = size
10:
11: function DEALLOC_INSTRUM(ptr)
12:   memory.dealloc(ptr)
13:
14: function MEM_WRITE_INSTRUM(addr, length)
15:   shadow.unpoison(addr:addr+length)
16:   memory.mem_write(addr)
17:
18: function MEM_READ_INSTRUM(addr, length)
19:   if not satisfiable(shadow.contains_unpoisoned(addr:addr+length)) then
20:     boom()
21:   memory.mem_read(addr)
```

As in previous sections, we do not repeat the clarifications already provided in Section 5.2.1. Instead, we emphasize the following additional aspects:

- Here again, when initialising we unpoison all the regions outside of the heap since those are not of interest for this algorithm.
- For allocations the entire allocated buffer is poisoned, as no initialised data is expected to reside in newly allocated memory. But upon memory deallocation the freed region is not poisoned again. This design choice is motivated by the fact that this algorithm does not handle accesses to unallocated memory; such invalid accesses are detected by a separate algorithm. Consequently, the poisoned state of deallocated memory is irrelevant in this context.
- Any write operation unpoisons the corresponding memory region.
- For intercepted memory reads, the instrumentation verifies whether the entire read operation targets poisoned memory and doesn't check whether it may only partially read poisoned memory. This is because load widening optimizations may cause reads of initialised memory to be widened, thereby partially overlapping with uninitialised memory. So we conservatively assume that a read that only partially reads poisoned memory is not a vulnerability. A violation is reported only if the entire read originates from uninitialised memory.

5.3 Stack

5.3.1 Return Address

This section describes the pseudocode for detecting memory accesses to the stored return address that are not caused by `ret` instructions and are therefore typically illegal. The pseudocode is presented in Algorithm 5. This mechanism allows us to detect certain stack buffer overflows as well as control-hijacking attempts.

The algorithm poisons the memory location holding the return address at every `call` instruction and unpoisons it at every corresponding `ret` instruction. At every access which is not caused by a return, it reports the vulnerability. In addition, at each `ret`, it verifies that the return is performed while `rsp` points to the expected return address

Algorithm 5 Detecting illegal return address access.

```

1: function INIT( ):
2:   shadow.unpoison(1, memory.max_addr)
3:
4: function CALL_INSTRUM(addr)
5:   memory.call(addr)
6:   rsp_after_call = memory.rsp
7:   shadow.poison(rsp_after_call:rsp_after_call+7)
8:   ret_stack.push(rsp_after_call)
9:
10: function RET_INSTRUM( )
11:   rsp_before_ret = memory.rsp
12:   rsp_after_call = ret_stack.pop()
13:   if satisfiable(rsp_before_ret != rsp_after_call) then
14:     boom()
15:   shadow.unpoison(rsp_before_ret:rsp_before_ret+7)
16:   memory.ret()
17:
18: function MEM_ACCESS_INSTRUM(addr, length)
19:   if satisfiable(shadow.contains_poisoned(addr:addr+length)) then
20:     boom()
21:   memory.mem_access(addr, length)

```

The code is straight-forward. After a call instruction, `rsp` points at the return address which is the reason why the location pointer by `rsp` is poisoned and pushed in the `ret_stack` data structure. As shown in the memory access instrumentation, detecting invalid accesses to the return address itself is straightforward.

During execution of `ret` in `ret_instrum()` the value of `rsp` is compared with the value of `ret_stack.pop()` to check whether the return is executed while `rsp` points at the correct address.

Furthermore, the function `mem_access_instrum()` instruments every memory access except those performed by the `call` and `ret` instructions themselves.

5.3.2 Red Zone

This section describes the pseudocode for detecting memory accesses below the red zone, which is defined as the 128-byte region below `rsp` where memory accesses are still permitted for leaf functions. The pseudocode is shown in Algorithm 6.

Algorithm 6 Detecting stack access below the red zone.

```

1: function MEM_ACCESS_INSTRUM(addr, length)
2:   if satisfiable((addr:addr+length)  $\cap$  (memory.stack_bound:memory.rsp-128)  $\neq \emptyset$ ) then
3:     boom()
4:   memory.mem_access(addr, length)

```

As can be seen, we do not use shadow memory for this check. While a shadow memory-based implementation would be possible, it would merely replicate the behavior implemented here, while additionally requiring storage for the shadow stack.

It should be noted that memory access instrumentation must not be applied to exception handlers or debuggers, as these are permitted to access memory below the red zone.

5.3.3 Uninitialised Stack Memory or Use-after-return

This section describes the pseudocode for detecting accesses to stack memory that should be considered uninitialised. The pseudocode is shown in Algorithm 7. The algorithm checks memory reads on the stack to determine whether the accessed memory has been initialised (i.e., written to previously), thereby detecting uninitialised-read violations. Additionally, after every `call` and `ret` instruction, it resets the initialisation state of all stack memory below `rsp` to uninitialised. Doing this after every `call` instruction is necessary because, even if a caller has written data into the red zone, the callee must not rely on that data. Doing this after every `ret` instruction enables detection of use-after-return violations.

Algorithm 7 Detecting access to uninitialised stack memory or use-after-return.

```
1: function INIT( )
2:   shadow.unpoison(1, memory.max_addr)
3: function WRITE_TO_RSP_INSTRUM(new)
4:   old = memory.rsp
5:   if new < old then shadow.poison(new-128:old-128)
6:   else if new > old then shadow.unpoison(old-128:new-128)
7:   memory.rsp = new
8: function CALL_INSTRUM(addr)
9:   memory.call(addr)
10:  rsp = memory.rsp
11:  shadow.poison(rsp-128:rsp)
12: function RET_INSTRUM( )
13:  shadow.unpoison(rsp-128:rsp)
14:  memory.ret()
15:  shadow.poison(rsp-128:rsp)
16: function MEM_READ_INSTRUM(addr, length)
17:   if not satisfiable(shadow.contains_unpoisoned(addr:addr+length)) then boom()
18:   memory.mem_read(addr, length)
19: function MEM_WRITE_INSTRUM(addr, length)
20:   if satisfiable((addr:addr+length)  $\cap$  (memory.rsp-128:rsp_after_first_call))  $\neq \emptyset$  then
21:     shadow.unpoison(addr:addr+length)
```

We clarify the algorithm:

- After `init()`, which unpoisons the entire memory, the first intercepted event is the call to the program's initial function. This causes the region from `rsp-128` to `rsp` (the red zone) to be poisoned, as it is considered uninitialised.
- Subsequent modifications to `rsp`, intercepted by `write_to_rsp_instrum()`, update the poisoned region accordingly. If `rsp` is decremented (i.e., the stack grows and the red zone shifts downward), the newly exposed stack memory, which is uninitialised, is poisoned. If `rsp` is incremented, the lowest portion removed from the red zone is unpoisoned, as it is no longer tracked by this algorithm. We assume that `write_to_rsp_instrum()` intercepts all writes to `rsp` except those performed by `call` and `ret` instructions.

- For intercepted memory reads, the instrumentation verifies whether the entire read operation targets poisoned memory and doesn't check whether it may only partially read poisoned memory. This is because load widening optimizations may cause reads of initialised memory to be widened, thereby partially overlapping with uninitialised memory. So we conservatively assume that a read that only partially reads poisoned memory is not a vulnerability. A violation is reported only if the entire read originates from uninitialised memory. We assume that `mem_read_instrum()` and `mem_write_instrum()` also intercept `call` and `ret` instructions.
- As shown in the memory write instrumentation, any memory location written to is subsequently unpoisoned, as it is no longer considered uninitialised.

5.3.4 Longjmp, Exceptions or Cloning

As discussed in Section 2.3.2.6, long jumps, C++ exceptions, and cloning require special handling. In this section, we propose an approach for handling long jumps; exception handling and cloning can be addressed analogously.

Algorithm 8 Handling `longjmp`

```

1: function LONGJMP_INSTRUM(env, val)
2:   rsp_before = memory.rsp
3:   memory.longjmp(env, val)
4:   rsp_after = memory.rsp
5:   rbp_after = memory.rbp
6:   found = False
7:   while not found do
8:     found = not satisfiable(rbp_after != ret_stack.pop())
9:   if not found then
10:    boom()
11:
12:   // Handle return address
13:   shadow.unpoison(rsp_before:rbp_after-1)
14:
15:   // Handle uninitialised memory
16:   shadow.unpoison(rsp_before-128:rsp_after-128)

```

First, we verify whether the `longjmp` operation is valid, i.e., whether it restores a state in which `rbp` points to a correct return address. To perform this check, we reuse `ret_stack`, which is also employed by the algorithm that ensures safe access to return addresses. Since `ret_stack` stores the locations of return addresses, we compare the restored `rbp` value against the addresses popped from the stack. A match is considered found if it is unsatisfiable that `rbp` and a popped address are unequal i.e. if they are equal in any model of the constraint solver.

Once the validity check succeeds, we update the shadow memory. We inserted the code for the two relevant algorithms: the return-address handling algorithm and the uninitialised-memory handling algorithm. For return addresses, we unpoison all return addresses that have been skipped by the long jump. For uninitialised memory, we unpoison all memory below the red zone. We conservatively didn't poison the red zone after performing the long jump since we don't know what assumptions the user applying the `longjmp` does, but poisoning it might be a valid modification to our algorithm. Or poisoning it in cases when there have been function calls whose returns were skipped by the call to `longjmp` might also be valid.

5.3.5 Why We Do Not Consider Callee-Preserved Registers and Object Boundaries

Analogously to the return-address algorithm described in Section 5.3.1, it would be possible to implement shadow memory to track all callee-preserved registers. However, as discussed in Section 2.3.2.4, a shadow stack as defined there provides a significantly simpler solution. Since the principle of address sanitisation has already been demonstrated for return addresses, we consider it unnecessary to present a separate algorithm for callee-preserved registers.

Furthermore, this thesis does not focus on reverse engineering. Without reverse-engineering techniques, identifying object boundaries within a stack frame is difficult, if not impossible. Consequently, we do not include algorithms for detecting overflows across object boundaries on the stack.

Chapter 6

Data Structures and Design

In this chapter, we present several data structures and algorithms that can be used to implement shadow memory. Section 6.1 introduces the problem space and motivates why a naive approach is not feasible for shadow memory implementation. In the course of this discussion, we introduce two orthogonal dimensions along which shadow memory implementations can be categorised.

Section 6.2 then present the corresponding shadow memory implementations, together with an analysis of their runtime complexity, space consumption, and the number of solver queries required. Finally, Section 6.3 discusses which shadow memory implementation is suitable for each detection algorithm introduced in Section 5.

6.1 Overview

We start with an introduction to the topic in Section 6.1.1 which, for simplicity reasons, won't consider symbolic execution. Then in Section 6.1.2 we will additionally consider all the relevant aspects that come with symbolic execution.

6.1.1 The Concrete Case

A fundamental design question concerns the implementation of shadow memory. Conceptually, shadow memory partitions the address space into poisoned and unpoisoned regions. A straightforward and naive implementation uses a dictionary in which keys correspond to memory addresses and values indicate whether the respective address is poisoned. An equivalent alternative is a set-based representation that stores either all poisoned or all unpoisoned addresses. We refer to implementations of this kind as *address-based shadow memory*.

A defining characteristic of address-based shadow memory is that updating the poisoned state of a single address has time complexity $O(1)$; consequently, updating n addresses requires $O(n)$ time. (Although practical implementations of sets and dictionaries do not guarantee strict $O(1)$ complexity in all cases, they typically achieve constant-time performance on average. For simplicity, we assume $O(1)$ complexity throughout this discussion.) Similarly, checking whether a given address is poisoned takes $O(1)$ time, or $O(n)$ for checking n addresses. The space complexity is $O(1)$ per address and $O(n)$ for storing information about n addresses.

Assuming a fully concrete execution, address-based shadow memory provides a fast implementation but incurs high space overhead, as it requires storing state information for each address (or at least for each poisoned or unpoisoned address). In theory, since each address may be poisoned or unpoisoned independently of all others, this per-address storage requirement is unavoidable. However, in practice, poisoned addresses typically form contiguous regions. In such cases, a more space-efficient representation stores only the beginning and size of each poisoned or unpoisoned chunk. For large chunk sizes, this representation can significantly reduce

space usage. Assuming an average chunk size of $size$ addresses, the amortized space cost per address becomes $O(\frac{1}{size})$. We refer to implementations following this approach as *range-based shadow memory*.

This space efficiency of range-based shadow memories comes at the cost of increased runtime complexity. Representing chunk boundaries requires maintaining an ordered data structure, such as a binary search tree. Assuming n chunks, insertion and lookup operations have time complexity $O(\log n)$. Several data structures are applicable in this context, including binary search trees, balanced trees, and red-black trees.

6.1.2 The Symbolic Case

When symbolic execution is considered, the situation becomes more complex. Let us denote with *shadow chunks* the contiguous areas of the shadow memory which are poisoned or unpoisoned. Let us further denote with *checked address* the address for which it is checked whether it accesses poisoned memory. When considering symbolic execution, three elements that are typically concrete may become symbolic: shadow chunk beginnings, shadow chunk sizes, and checked addresses. Allowing those three to become symbolic requires modifications on both address-based shadow memories and range-based shadow memories which also affect time and space performance.

Before we discuss the modifications as well as the runtime and space complexities, we will first argue that while we will support symbolic shadow chunk sizes and symbolic checked addresses in our implementations of symbolic memory, symbolic shadow chunk beginnings can be excluded from consideration. This claim is justified by analyzing the specific address sanitisation algorithms evaluated in this work which are listed here:

- The address sanitising algorithm for checking correct heap memory management, introduced in Section 5.2.1, denoted as HMM (*Heap Memory Management*).
- The address sanitising algorithm for checking correct heap memory usage, introduced in Section 5.2.2, denoted as HMU (*Heap Memory Usage*).
- The address sanitising algorithm for detecting uninitialised reads from heap memory, introduced in Section 5.2.3, denoted as HRI (*Heap Read initialised*).
- The address sanitising algorithm for detecting invalid accesses to stored return addresses, introduced in Section 5.3.1, denoted as RA (*Return Address*).
- The address sanitising algorithm for detecting uninitialised reads from stack memory and use-after-return errors, introduced in Section 5.3.3, denoted as SRI (*Stack Read initialised*).

We now analyze for each of those address sanitising algorithms which of the three elements (shadow chunk beginning, shadow chunk size, checked address) can be symbolic.

- HMM: The only unpoisoned shadow chunks are single addresses corresponding to the beginnings of allocated buffers. Consequently, unpoisoned shadow chunk sizes are always 1 and not symbolic. Allocated buffer beginnings are concrete as discussed in 4.4.2, so unpoisoned shadow chunk beginnings are also concrete. Since unpoisoned shadow chunk beginnings and sizes are concrete, poisoned shadow chunk beginnings and sizes must be as well. However, checked addresses - which are the pointers passed to deallocating functions - may be symbolic.
- HMU: The only unpoisoned shadow chunks correspond to allocated buffers and those have concrete beginnings, thus unpoisoned shadow chunks also have concrete beginnings. The allocated buffer sizes are concretised to the maximum as discussed in 4.4.2. But as mentioned there, those allocated buffers rather represent reservations of memory for the buffer, not the semantically relevant buffer sizes. The semantically relevant buffer sizes remain symbolic. So, the size of unpoisoned shadow chunks is symbolic. Since poisoned chunks begin where unpoisoned chunks end, the beginnings as well as the sizes of the poisoned chunks will be symbolic as well. Checked addresses might be symbolic as well.
- HRI: Unpoisoned shadow chunks correspond to memory regions that have been written to. As discussed in 4.4.3, both the location and size of each write are concretised, making unpoisoned shadow chunk beginnings and sizes concrete. Consequently, poisoned shadow chunk beginning and sizes are also concrete. Checked addresses which correspond to memory reads may be symbolic.

- RA: Poisoned shadow chunks are fixed-size (8-byte) regions storing return addresses. Poisoned chunk sizes are therefore always concrete, and poisoned chunk beginnings are concrete because all memory writes — including return address stores — are concretised. Therefore also unpoisoned chunk begins and sizes are concrete. Memory access addresses may again be symbolic.
- SRI: As in HRI, both shadow chunk beginnings and chunk sizes are concrete. Only checked addresses might be symbolic.

From this analysis, we conclude that we have two models regarding the elements that are concrete or symbolic:

- Shadow memory where chunk begins and chunk sizes are both concrete but where checked addresses may be symbolic. This is the case for HMM, HRI, RA and SRI. We call this model *concrete-chunk* shadow memory.
- Shadow memory where unpoisoned chunk begins are concrete, and unpoisoned chunk sizes as well as poisoned chunk begins, poisoned chunk sizes and checked addresses may be symbolic. And every chunk that is to be unpoisoned does not overlap with an already unpoisoned chunk nor does it start at the end of an existing unpoisoned chunk. This is the case for HMU. We call this model *symbolic-chunk* shadow memory.

In Section 6.2.2, we present pseudocode for both concrete-chunk address-based shadow memory and symbolic-chunk address-based shadow memory, and provide a detailed analysis of its runtime, space consumption, and solver query complexity. Section 6.2.3 provides an analogous treatment for range-based shadow memory. Finally, Section 6.3 applies this analysis to determine the appropriate shadow memory representation for each of the five address sanitisation algorithms discussed above.

6.2 Shadow Memory Implementations

In this section, we present the different shadow memory implementations.

6.2.1 Remarks on Pseudocode and on Notation for Analysis

It must be noted that the shadow memory implementations presented here don't adhere to the shadow memory interfaces shown in Chapter 5 introducing the detection semantics. The reason for this is that using an interface as presented in mentioned chapter would not allow for an efficient implementation, but enables good visualisation of the algorithms. One change is e.g. that the solver is not invoked after the shadow memory functions (like `shadow.is_poisoned()`) return a symbolic Boolean value, but is already used inside the shadow memory so that the shadow memory can return a concrete Boolean value.

For the subsequent time, space and solver query analysis, we introduce the following notation:

- Checked addresses may be concrete or symbolic. If an address is symbolic, it may have multiple concretisations. We denote by $\#addr$ the average number of possible concretisations of checked addresses. We further denote by $range(addr)$ the average span of these concretisations, defined as $max(addr) - min(addr)$. Finally, we denote with no_acc the total number of address checks performed until program termination or the detection of a vulnerability.
- Let no_u_ch denote the average number of unpoisoned chunks present during execution. We denote by ch_add the number of chunks that have become unpoisoned, and by ch_rem the number of chunks that have become poisoned again, both measured up to program termination or vulnerability detection. Furthermore, let u_ch_size denote the average maximum size of unpoisoned chunks, p_ch_size the average maximum size of chunks that are poisoned again (which is not the average maximum size of all chunks that are poisoned but only those that are poisoned due to a call to `shadow.poison()`), and ch_size the average maximum size of all shadow chunks between the first and the last unpoisoned chunk.
- We assume that a solver query takes $O(1)$ time to obtain a single concretisation of $addr$. If the solver is required to enumerate all $\#addr$ concretisations, we assume a time complexity of $O(\#addr)$. For the purpose of solver-query counting, such an enumeration is treated as $\#addr$ separate solver queries.

- In analyzing solver queries, we assume a worst-case scenario in which every value that may be symbolic is indeed symbolic. In practice, non-symbolic expressions would typically be evaluated directly without invoking the solver, as solver queries are comparatively expensive. However, to avoid assuming any specific distribution of symbolic versus concrete values, we adopt this conservative worst-case assumption throughout the analysis.

6.2.2 Address-based Shadow Memory

This section covers both the concrete-chunk and symbolic-chunk implementations of address-based shadow memory.

6.2.2.1 Concrete-chunk Implementation

The concrete-chunk address-based shadow memory implementation is shown in Algorithm 9. It employs a set as its underlying data structure and explicitly stores all unpoisoned addresses. As a result, the implementations of `shadow.poison()`, `shadow.unpoison()`, and `shadow.is_poisoned()` are straightforward. In this model, both the beginnings and sizes of unpoisoned shadow chunks are concrete, while the addresses being checked may be symbolic.

Algorithm 9 Concrete-chunk Address-based Shadow Memory Implementation

```

1: function SHADOW.UNPOISON(begin, size)
2:   for i in range(begin, begin + size) do
3:     set.add(i)
4:
5: function SHADOW.POISON(begin, size)
6:   for i in range(begin, begin + size) do
7:     set.remove(i)
8:
9: function SHADOW.IS_POISONED(addr)
10:  conc_addrs = concretise(addr)
11:  for conc_addr in conc_addrs do
12:    if conc_addr not in set then
13:      boom()

```

We now analyze the runtime, space, and solver query complexity of the concrete-chunk address-based shadow memory:

- **Time:** The `unpoison` operation has time complexity $O(u_ch_size)$, while the `poison` operation has time complexity $O(p_ch_size)$. The `is_poisoned` operation has time complexity $O(\#addr)$, as it requires $O(\#addr)$ time for the solver to enumerate all concretisations of the checked address and an additional $O(\#addr)$ time to verify that none of these concretisations is contained in the set of unpoisoned addresses. Consequently, the overall time complexity is

$$O(ch_add \cdot u_ch_size + ch_rem \cdot p_ch_size + no_acc \cdot \#addr).$$

- **Space:** The space complexity is $O(no_u_ch \cdot u_ch_size)$, corresponding to the storage required for all unpoisoned addresses.
- **Number of solver queries:** The `poison` and `unpoison` operations do not issue solver queries, yielding a complexity of $O(0)$ for these operations. The `is_poisoned` operation issues $O(\#addr)$ solver queries. Overall, the number of solver queries is

$$O(no_acc \cdot \#addr).$$

6.2.2.2 Symbolic-chunk Implementation

The symbolic-chunk address-based shadow memory implementation is shown in algorithm 10. For each address belonging to an unpoisoned chunk with potentially symbolic size, the implementation inserts an entry into a dictionary, where the address serves as the key and the value is the chunk beginning. This is performed for every address in the range $(begin, begin + \max(size))$. In a separate dictionary *meta* we use the *begin* as key and store the size and the current maximal size.

When an address is checked, it is concretised to all possible values. For each such concretisation, the algorithm first determines whether it is possible for the address to lie within any unpoisoned chunk, that is, whether it falls within one of the ranges $(begin, begin + \max(size))$. If this is not the case, then no satisfying model exists in which the address is unpoisoned, and a vulnerability is reported.

If the concretised address does lie within such a range, the algorithm then checks whether it is satisfiable that the checked address is concretised to this specific value and that the resulting address exceeds $begin + size$. If this condition is satisfiable, then there exists at least one model in which the address is out of bounds, indicating a potential vulnerability.

Algorithm 10 Handling Symbolic-chunk address-based shadow memory implementation

```

1: function SHADOW.UNPOISON(begin, size)
2:   max_size = max(size)
3:   meta[begin] = (size, max_size)
4:   for i in range(begin, begin + max_size) do
5:     dict[i] = begin
6:
7: function SHADOW.POISON(begin)
8:   size, max_size = meta[begin]
9:   meta.remove(begin)
10:  for i in range(begin, begin + max_size) do
11:    dict.remove(i)
12:
13: function SHADOW.IS_POISONED(addr)
14:   conc_addrs = concretise(addr)
15:   for conc_addr in conc_addrs do
16:     if conc_addr not in dict.keys() then
17:       boom
18:     begin = dict[conc_addr]
19:     size, max_size = meta[begin]
20:     if satisfiable(begin + size <= conc_addr and addr == conc_addr) then:
21:       boom()

```

As already mentioned, during the unpoisoning operation we store, for each address in the range from *begin* to $begin + \max(size)$, the beginning address of the corresponding chunk. The value $\max(size)$ must be explicitly stored in the metadata structure *meta* alongside *size*, since it may not be reconstructible at a later point. During execution, the symbolic variable *size* may become concretised, in which case it is no longer possible to determine which addresses were originally affected by the unpoisoning operation. Therefore, when the chunk is poisoned again, the stored maximum size is used to identify and remove all addresses that were previously unpoisoned.

We now analyze the runtime, space, and solver query complexity of the symbolic-chunk address-based shadow memory:

- **Time:** The `unpoison` operation has time complexity $O(u_ch_size)$, while the `poison` operation has time complexity $O(p_ch_size)$. The `is_poisoned` operation has time complexity $O(\#addr)$, since it requires $O(\#addr)$ time for the solver to enumerate all concretisations of the checked address, $O(\#addr)$ time to verify whether these concretisations fall within any relevant range, and an additional $O(\#addr)$ time to determine — if they do fall within a range — whether there exists a model in which the address is out of bounds. Overall, this yields a total time complexity of

$$O(ch_add \cdot u_ch_size + ch_rem \cdot p_ch_size + no_acc \cdot \#addr).$$

- **Space:** The space complexity is $O(no_u_ch \cdot u_ch_size)$, reflecting the storage required for all unpoisoned shadow chunks.
- **Number of solver queries:** The `unpoison` operation issues $O(1)$ solver queries due to the invocation of `max`. The `poison` operation does not issue any solver queries, yielding a complexity of $O(0)$. The `is_poisoned` operation issues $O(\#addr)$ solver queries due to address concretisation and subsequent calls to `satisfiable`. Consequently, the overall number of solver queries is

$$O(ch_add + no_acc \cdot \#addr).$$

6.2.3 Range-based Shadow Memory

This section presents both the concrete-chunk and symbolic-chunk implementations of range-based shadow memory.

6.2.3.1 Concrete-chunk Implementation

The concrete-chunk range-based shadow memory implementation is shown in Algorithm 11. The implementation relies on an `ods` (ordered data structure), which supports insertion and removal operations in $O(\log n)$ time, where n is the number of stored entries, and which might be e.g. a balanced binary search tree.

For each unpoisoned shadow chunk, a node is created that stores the concrete beginning address and the concrete size of the chunk. Nodes are ordered by their beginning addresses. Each node additionally stores a pointer to its successor, a modification that does not significantly affect runtime or space complexity. The data structure also supports merging adjacent unpoisoned chunks; this optimization similarly does not materially affect asymptotic runtime or space requirements.

The data structure must additionally handle cases in which a newly inserted chunk overlaps with an existing chunk. While this increase the implementation complexity of `ods`, it does not significantly alter its asymptotic runtime or space behavior.

When checking an address `addr`, the algorithm first computes its minimum and maximum possible values. It then queries `ods` for the node whose beginning address is less than or equal to the minimum of `addr`. This query can be implemented in logarithmic time with respect to the number of entries in `ods`. The returned node, denoted `node`, contains the fields `begin`, `size`, and `next`.

The algorithm then checks whether it is satisfiable that

$$addr \geq node.begin + node.size \wedge addr < node.next.begin,$$

which corresponds to the address lying within the poisoned region between the unpoisoned chunk represented by `node` and its successor. If this condition is satisfiable, a vulnerability is reported. Otherwise, the algorithm follows the `next` pointer and repeats the check for the subsequent pair of chunks. This process continues until the algorithm exits the range of possible values of `addr`, that is, until it encounters a chunk whose end address exceeds the maximum of `addr`.

Algorithm 11 Handling Concrete-chunk range-based shadow memory implementation

```

1: function SHADOW.UNPOISON(begin, size)
2:   ods.insert(begin, size)
3:
4: function SHADOW.POISON(begin, size)
5:   ods.remove(begin, size)
6:
7: function SHADOW.IS_POISONED(addr)
8:   max_addr = max(addr)
9:   min_addr = min(addr)
10:  node = ods.get_smaller_equals(max_addr)
11:  if node == null then boom()
12:  next = node.next
13:  while True do:
14:    if max_addr < node.begin + node.size then return
15:    if next != null then
16:      if satisfiable(node.begin + node.size <= addr and addr < next.begin) then boom()
17:      node = next
18:      next = node.next
19:    else
20:      if satisfiable(node.begin + node.size <= addr) then boom()
21:      else
22:        return

```

We briefly explain the implementation of `is_poisoned()`:

- In line 9, the algorithm queries the unpoisoned chunk whose beginning address is less than or equal to the minimum of the checked address. If no such chunk exists, i.e., the minimum possible value of the checked address is smaller than the beginning of every unpoisoned chunk, a vulnerability is immediately detected.
- In line 13, the algorithm checks whether the iteration has left the range of the checked address. If this is the case and no vulnerability has been detected so far, the memory access is proven to be safe.
- In line 14, the algorithm handles the case where a successor chunk exists, i.e., the chunk stored in `node` is not the last unpoisoned chunk. It then checks whether it is satisfiable that the checked address lies within the poisoned region between the current chunk and its successor. If yes, a vulnerability has been found.
- In line 18, the algorithm considers the case where no successor chunk exists. In this situation, it checks whether the checked address can lie outside the bounds of the current chunk. If yes, a vulnerability has been found.

We now analyze the runtime, space, and solver query complexity:

- **Time:** The `poison` and `unpoison` operations each have a time complexity of $O(\log no_u_ch)$. The `is_poisoned` operation has a time complexity of

$$O\left(\log no_u_ch + \frac{range(addr)}{ch_size}\right).$$

The logarithmic term arises from querying the chunk whose beginning is less than or equal to the minimum possible value of the checked address. The fractional term reflects the cost of iterating over all chunks whose beginnings lie between the minimum and maximum possible values of the checked address. Consequently, the overall runtime complexity is

$$O\left(ch_add \cdot \log no_u_ch + ch_rem \cdot \log no_u_ch + no_acc \cdot \frac{range(addr)}{ch_size}\right).$$

- **Space:** The space complexity is $O(no_u_ch)$, since each unpoisoned chunk requires only a constant amount of stored metadata.
- **Number of solver queries:** The `poison` and `unpoison` operations do not issue any solver queries, yielding a complexity of $O(0)$. The `is_poisoned` operation issues $O\left(\frac{range(addr)}{ch_size}\right)$ solver queries, corresponding to the checks performed while iterating over relevant chunks. Thus, the overall number of solver queries is

$$O\left(no_acc \cdot \frac{range(addr)}{ch_size}\right).$$

6.2.3.2 Symbolic-chunk Implementation

The symbolic-chunk implementation uses the same pseudocode as the concrete-chunk variant. The primary difference is that chunk sizes are symbolic rather than concrete. Furthermore, for the address sanitisation algorithms considered in this thesis, it is not necessary to merge adjacent unpoisoned chunks, nor to handle cases in which a newly inserted chunk overlaps with an existing one. As a result, these differences do not affect the asymptotic runtime, space, or solver query complexity, which therefore remain unchanged.

6.3 Which one for which Purpose?

We now determine, for each address sanitisation algorithm, which shadow memory implementation is most appropriate.

6.3.1 Heap Memory Management

For HMM, both the concrete-chunk address-based and the concrete-chunk range-based implementations are viable candidates.

The runtime complexity of the address-based implementation is

$$O(ch_add \cdot u_ch_size + ch_rem \cdot p_ch_size + no_acc \cdot \#addr),$$

whereas the range-based implementation has runtime

$$O\left(ch_add \cdot \log no_u_ch + ch_rem \cdot \log no_u_ch + no_acc \cdot \frac{range(addr)}{ch_size}\right).$$

In HMM, we always poison or unpoison exactly one address, implying $u_ch_size = p_ch_size = 1$. The address-based runtime therefore simplifies to

$$O(ch_add + ch_rem + no_acc \cdot \#addr).$$

For the range-based implementation, note that as long as no vulnerability is detected, every concretisation of $addr$ must fall within an unpoisoned chunk. Consequently, there must be at least $\#addr$ unpoisoned chunks between $\min(addr)$ and $\max(addr)$, which implies an average chunk size ch_size of at least $\frac{range(addr)}{\#addr}$. Substituting this bound yields a runtime for the range-based shadow memory of at least

$$O(ch_add \cdot \log no_u_ch + ch_rem \cdot \log no_u_ch + no_acc \cdot \#addr).$$

Thus, the address-based implementation is strictly better in terms of runtime.

The space complexity of the address-based implementation is $O(no_u_ch \cdot u_ch_size)$, which simplifies to $O(no_u_ch)$, while the range-based implementation also requires $O(no_u_ch)$ space. Hence, neither approach is favored in terms of space.

The solver query complexity is $O(no_acc \cdot \#addr)$ for the address-based implementation and $O\left(no_acc \cdot \frac{range(addr)}{ch_size}\right)$ for the range-based one, which again simplifies to at least $O(no_acc \cdot \#addr)$. Thus, neither approach is favored with respect to solver queries.

Since space usage and solver query complexity are comparable, but the runtime is better for the address-based approach, we conclude that the concrete-chunk address-based shadow memory implementation is the most suitable choice for HMM.

6.3.2 Heap Memory Usage

For HMU, the choice lies between the symbolic-chunk address-based and symbolic-chunk range-based implementations. Of course it must be mentioned that for the address-based implementation, the regions between `memory.0` and `memory.heap_begin`, and the regions between `memory.heap_end` and `memory.max_addr` would not be all unpoisoned by adding their addresses in the set. This would be an unnecessary waste of space. Conceptually this would be implemented by executing every solver query with the additional constraint that the address lies within the heap bounds.

The runtime complexity of the address-based implementation is

$$O(ch_add \cdot u_ch_size + ch_rem \cdot p_ch_size + no_acc \cdot \#addr),$$

while the range-based implementation has runtime

$$O\left(ch_add \cdot \log no_u_ch + ch_rem \cdot \log no_u_ch + no_acc \cdot \frac{range(addr)}{ch_size}\right).$$

Since poisoned chunks correspond exactly to previously unpoisoned chunks, we have $u_ch_size = p_ch_size$. Furthermore, ch_add corresponds to the number of memory allocations and ch_rem to the number of deallocations. Let $no_op = ch_add + ch_rem$ denote the total number of memory management operations. The runtimes simplify to

$$O(no_op \cdot p_ch_size + no_acc \cdot \#addr)$$

for the address-based implementation and

$$O\left(no_op \cdot \log no_u_ch + no_acc \cdot \frac{range(addr)}{ch_size}\right)$$

for the range-based implementation.

Assuming that memory accesses occur significantly more frequently than memory management operations, the dominant terms are those involving no_acc . This yields $O(no_acc \cdot \#addr)$ for the address-based and $O\left(no_acc \cdot \frac{range(addr)}{ch_size}\right)$ for the range-based implementation. Since we could not find any literature on the distribution for concretisations of symbolic addresses, it is unclear which of these two expressions is smaller in practice.

If one assumes that concretisations of symbolic addresses typically lie within contiguous address ranges, then $O\left(no_acc \cdot \frac{range(addr)}{ch_size}\right)$ may often be close to $O(no_acc)$, favoring the range-based implementation. However, this assumption cannot be justified rigorously.

The space complexity is $O(no_u_ch \cdot u_ch_size)$ for the address-based implementation and $O(no_u_ch)$ for the range-based implementation, clearly favoring the latter.

Solver query complexity is $O(ch_add + no_acc \cdot \#addr)$ for the address-based implementation and $O\left(no_acc \cdot \frac{range(addr)}{ch_size}\right)$ for the range-based one. Again, none can be clearly favored.

We therefore cannot conclusively determine the superior implementation for HMU, but we believe that the symbolic-chunk range-based shadow memory implementation is likely the better choice in practice.

6.3.3 Uninitialised Heap Memory

For HRI, both concrete-chunk address-based and concrete-chunk range-based implementations are viable candidates. Here it must be mentioned again that for the address-based implementation, the regions between `memory.0` and `memory.heap_begin`, and the regions between `memory.heap_end` and `memory.max_addr` would not be all unpoisoned by adding their addresses in the dictionary. A similar approach as in HMU by adding constraints restricting the checked address to lie within the heap would be used for every solver query.

The runtime complexity of the address-based implementation is

$$O(ch_add \cdot u_ch_size + ch_rem \cdot p_ch_size + no_acc \cdot \#addr),$$

whereas the range-based implementation has runtime

$$O\left(ch_add \cdot \log no_u_ch + ch_rem \cdot \log no_u_ch + no_acc \cdot \frac{range(addr)}{ch_size}\right).$$

In the context of HRI, ch_add aggregates the number of `calloc` calls, memory writes, and certain `realloc` operations, while ch_rem aggregates `malloc`, `free`, and other `realloc` operations, and while no_acc aggregates the number of reads. Although algorithm 14 does not poison memory on `free`, doing so in practice would reduce the size of the data structure.

Since reads and writes occur far more frequently than memory management operations, we simplify ch_add to the number of writes, ch_rem to zero, and no_acc to the number of reads. This yields a runtime of

$$O(ch_add \cdot u_ch_size + no_acc \cdot \#addr)$$

for the address-based implementation and

$$O\left(ch_add \cdot \log no_u_ch + no_acc \cdot \frac{range(addr)}{ch_size}\right)$$

for the range-based one.

Neither runtime expression can be shown to dominate the other. While u_ch_size (the size of reads) is typically small (often between 1 and 8 bytes), favoring the address-based implementation, the same arguments regarding $\#addr$ versus $\frac{range(addr)}{ch_size}$ apply as in the HMU case, favoring the range-based implementation.

The space complexity favors the range-based implementation, with $O(no_u_ch)$ versus $O(no_u_ch \cdot u_ch_size)$ for the address-based approach.

The solver query complexity is $O(no_acc \cdot \#addr)$ for the address-based implementation, and $O\left(no_acc \cdot \frac{range(addr)}{ch_size}\right)$ for the range-based implementation. Again, we cannot favour one.

We therefore conclude that, while no definitive choice can be made, the concrete-chunk range-based shadow memory implementation is likely the preferable option for HRI.

6.3.4 Return Address

The RA algorithm can be implemented analogously to HMM, with the only difference being that $u_ch_size = p_ch_size = 8$ instead of 1, and that the data structures keep not track of the unpoisoned, but the poisoned addresses. The same arguments therefore apply, and we conclude that the concrete-chunk address-based implementation is the most suitable choice for RA.

6.3.5 Uninitialised Stack Memory and Use-after-return

For SRI, both concrete-chunk address-based and concrete-chunk range-based implementations are again viable options.

As in the HRI case, ch_add aggregates several events but can be approximated as the number of writes, ch_rem can be treated as zero, and no_acc corresponds to the number of reads. This yields the same simplified runtime expressions as for HRI. Consequently, the same reasoning applies.

We therefore reach the same conclusion: while no definitive choice can be made, the concrete-chunk range-based shadow memory implementation appears to be the more appropriate option for SRI.

Chapter 7

Compatibility with Path Explosion Mitigation Techniques

7.1 Overview

In Section 2.1.4 we introduced several mitigation techniques which address the path explosion problem. In this section, we discuss which of these techniques can be combined with symbolic execution, address sanitisation, and binary-only analysis.

Table 7.1 summarises which mitigation technique might introduce unsoundness or incompleteness (row *incorrectness*) or might be incompatible with our theoretical approach (row *incompatible*). This table should be interpreted with caution. Accurately classifying each technique would require an in-depth analysis of the respective approach, and for several techniques the classification strongly depends on the concrete implementation. The table therefore serves only as a high-level overview of our assessment and is not discussed exhaustively.

Table 7.1: Compatibility of path explosion mitigation techniques with combination of symbolic execution, address sanitisation and binary-only analysis.

solution	incorrectness	incompatible
Program or environment modelling	x	
Pruning infeasible paths		
Path prioritisation: simple heuristics		
Path prioritisation: Bug-tailored heuristics		
Path prioritisation: Heuristics that deal with symbolic loop conditions		
Path prioritisation: Taint analysis		
Path prioritisation: Program slicing		
Underconstrained symbolic execution	x	
Preconditioned symbolic execution		
State merging	x	

It is encouraging that all mitigation techniques listed above are, in principle, compatible with our approach. We came to the conclusion that they are all compatible with our approach because after examination of those mitigation techniques we didn't find any issue that could not be solved with reasonable effort. In the remainder of this section, we therefore focus on techniques that introduce incorrectness.

It should be noted that all path prioritisation techniques introduce unsoundness if they do not just prioritise certain paths but instead discard others entirely. Similarly, preconditioned symbolic execution may introduce unsoundness or incompleteness if the specified preconditions are invalid, and underconstrained symbolic execution naturally produces false positives since it is not executed in the program’s context. However, since all of those incorrectness sources are not specific to address sanitisation, we don’t discuss them further here.

Program or environment modelling, as already observed in the context of ANGR `SIMPROCEDURES`, often introduces incorrectness due to simplified behaviour. Since these techniques are not specific to address sanitisation, and the limitations of the `SIMPROCEDURES` for memory management have already been discussed in 4.4.2, we do not consider them further here.

As discussed in 2.1.4, underconstrained symbolic execution may be applied by symbolically executing only a fragment of a program, such as a single function, rather than the entire program. In this setting, the function may receive complex objects as inputs. Depending on the chosen strategy, only those fields that are actually accessed are initialised lazily, and we previously introduced four possible ways in which such fields may be initialised. However, memory buffer boundaries that are not associated with heap buffers - as well as beginnings of heap buffers - must be concretised eagerly when using the default memory model of ANGR. As a result, the shadow memory must be constructed eagerly as well. So, together with ANGR’s default memory model, underconstrained symbolic execution introduces unsoundness by not being able to be underconstrained but rather overconstrained when having memory buffers as arguments.

We discuss state merging in more detail in the following subsection.

7.2 State Merging

As discussed in section 2.1.4, state merging associates each state to be merged with a corresponding condition, which we refer to here as the *state condition*. State merging is already implemented in ANGR for its main memory model. For shadow memory, however, the implementation of state merging depends both on the chosen shadow memory representation (one of the four variants proposed earlier) and on whether sound and complete merging (as implemented by default in ANGR) or an overapproximating merging strategy is employed. Since the design of shadow memory merging under overapproximation strongly depends on the specific approximation strategy, we do not consider it further here. Instead, we focus exclusively on sound and complete state merging.

While the concept of sound and complete state merging is conceptually straightforward, its implementation is often more complex. In particular, runtime overhead, solver complexity, and code complexity may increase significantly. Determining whether such an implementation is desirable in practice would require either a proof-of-concept implementation or deeper domain-specific knowledge. Nevertheless, we present the theoretical foundations in the following.

We now revisit the four proposed shadow memory implementations and discuss the required modifications to their algorithms and data structures. We then propose merging strategies to support sound and complete state merging.

7.2.1 Concrete-chunk Address-based Shadow Memory

To support state merging in the concrete-chunk address-based shadow memory, sets must be replaced by dictionaries. Suppose a concrete address is unpoisoned under a state condition c . Instead of executing `set.add(addr)`, we assign `dict[addr] = c` if `addr` is not yet present in the dictionary’s keys, or update it to `dict[addr] = dict[addr]  $\vee$  c` otherwise. The underlying idea is that every shadow memory operation must be interpreted as being performed only under the current state condition. For example, if a state is forked into states `this` and `other`, and only `this` unpoisons a particular address, then after merging, the address is unpoisoned

only when c_{this} holds. Consequently, the shadow memory must associate each address with the condition under which it is unpoisoned.

Following this principle, checking whether an address $addr$ is poisoned under a state condition c proceeds as follows. The symbolic address $addr$ is concretised to a value $conc_addr$. Rather than checking set membership, we first test whether $conc_addr$ is a key in the dictionary. This is required since if it is not a key in the dictionary, there is no possible model in the solver that makes this address unpoisoned, and we have found a vulnerability. If it is in the keys, we check whether it is satisfiable that both $addr = conc_addr$ and the stored condition hold simultaneously, i.e., all together we check

$$conc_addr \in \text{dict.keys}() \wedge \text{satisfiable}(addr = conc_addr \wedge \text{dict}[conc_addr]).$$

We now describe the merging strategy. Let $\text{dict}_{\text{this}}$ and $\text{dict}_{\text{other}}$ be the dictionaries of the two states, and let $\text{dict}_{\text{merged}}$ be the merged dictionary. For each address $addr$, we compute the merged condition i.e. the condition that must hold such that the address is unpoisoned in either of the states

$$c_{\text{merged},addr} = \text{dict}_{\text{this}}[addr] \vee \text{dict}_{\text{other}}[addr],$$

and store it as $\text{dict}_{\text{merged}}[addr] = c_{\text{merged},addr}$. If an address does not occur in one of the dictionaries, the corresponding condition is treated as `False`. If $c_{\text{merged},addr}$ is `False`, no entry is required in the merged dictionary since the address was not unpoisoned in either state.

Note that this approach is intentionally unoptimised for clarity. For example, during unpoisoning one could initially record `True` as the condition and defer the introduction of explicit state conditions until an actual merge occurs.

7.2.2 Symbolic-chunk Address-based Shadow Memory

To explain state merging for the symbolic-chunk address-based shadow memory, we first simplify the original algorithm. We assume that the auxiliary `meta` structure is eliminated and that all information is stored in a single dictionary of the form $\text{dict}[addr] = (\text{begin}, \text{size}, \text{max_size})$. This simplification serves only to clarify the merging principle.

To support merging, we replace dict with two dictionaries. The first, $\text{dict}_p[addr]$ (where p denotes *poisoned*), maps each address to a set of tuples $(\text{begin}, \text{size}, \text{max_size})$, similarly to the original dictionary but with the difference that not only one triple is stored but a whole set of triples. The second, $\text{dict}_c[(addr, \text{begin}, \text{size}, \text{max_size})]$ (where c denotes *condition*), maps quadruples to the conditions under which the address $addr$ potentially belongs to a chunk starting at begin with size size and maximum size max_size .

When unpoisoning a chunk with beginning begin and symbolic size size under state condition c , the following steps are performed for each address $addr$ in the range $[\text{begin}, \text{begin} + \max(\text{size}))$:

- If $\text{dict}_p[addr]$ is undefined, initialise it to the set containing only $(\text{begin}, \text{size}, \max(\text{size}))$. Else add $(\text{begin}, \text{size}, \max(\text{size}))$ to it.
- If $\text{dict}_c[(addr, \text{begin}, \text{size}, \max(\text{size}))]$ is undefined, initialise it to c . Else add c via logical disjunction.

To check whether an address $addr$ is unpoisoned, we first concretise it to $conc_addr$. The original procedure is replaced by the following:

- Check whether $conc_addr \in \text{dict}_p.\text{keys}()$. If not, report an error.
- Retrieve all tuples $(\text{begin}_i, \text{size}_i, \text{max_size}_i)$ from $\text{dict}_p[conc_addr]$ and their corresponding conditions $c_i = \text{dict}_c[(conc_addr, \text{begin}_i, \text{size}_i, \text{max_size}_i)]$.
- Check

$$\text{satisfiable}\left(addr = conc_addr \wedge \bigvee_i (c_i \wedge \text{begin}_i + \text{size}_i < conc_addr)\right).$$

This final check determines whether there exists a model in which $addr$ equals $conc_addr$ and lies outside the bounds of at least one unpoisoned chunk under the corresponding condition. If that check succeeds, report an error.

For merging, let $(dict_{p,this}, dict_{c,this})$ and $(dict_{p,other}, dict_{c,other})$ be the shadow memory structures of the two states which are to be merged into $(dict_{p,merged}, dict_{c,merged})$. The merged structures are constructed as follows. For each address $addr$, the union of the sets is stored into $dict_{p,merged}[addr]$:

$$dict_{p,merged}[addr] = dict_{p,this}[addr] \cup dict_{p,other}[addr].$$

For each quadruple $(addr, begin, size, max_size)$, we define the condition that must be true such that $addr$ is potentially in a buffer beginning at $begin$ with size $size$ and maximum size max_size by taking the disjunction of the conditions of the two states:

$$c_{merged} = dict_{c,this}[(addr, begin, size, max_size)] \vee dict_{c,other}[(addr, begin, size, max_size)],$$

We store it in $dict_{c,merged}$ if non-False. As before, missing entries of the single dictionaries are treated as False, and if c_{merged} is False no entry is created in $dict_{c,merged}$.

This algorithm is again presented in an unoptimised form for conceptual clarity.

7.2.3 Range-based Shadow Memory

We did not develop a concrete algorithm for merging range-based shadow memory, as such an algorithm would be substantially more complex than its address-based counterpart and would exceed the scope of this thesis. In particular, a sound treatment would require additional foundations in mathematical logic and symbolic data-structure manipulation. Nevertheless, we provide an outline of how such a merging mechanism might be realised. We emphasise that the following discussion is exploratory in nature and should be understood as an intuition for a possible implementation rather than a verified or complete solution.

At present, both range-based shadow memory implementations associate each node with three values: *begin*, *size*, and *next*. To support state merging, each node would need to store the values *begin*, *chunk_set*, and *chunk_cond*. The field *chunk_set* is a set of tuples of the form $(size, next)$, representing possible chunks starting at *begin*. The field *chunk_cond* is a dictionary mapping each tuple $(size, next)$ to a condition that characterises under which circumstances a chunk beginning at *begin* has size *size* and has *next* as successor.

We do not describe here how poisoning, unpoisoning, or access checks would be implemented in general. Instead, we illustrate the intended behaviour using a concrete example.

Consider an initial state s_0 that is forked into two states s_1 and s_2 , which are subsequently merged into a state s_3 . Assume that s_0 unpoisons a buffer beginning at address b_0 , while s_1 additionally unpoisons a buffer beginning at b_1 and s_2 unpoisons a buffer beginning at b_2 , with $b_1 > b_0$ and $b_2 > b_0$. For simplicity, assume that the shadow memory is implemented as a binary search tree (BST). Immediately prior to merging, the BSTs corresponding to s_1 and s_2 are shown in Figures 7.1(a) and 7.1(b), respectively where the placement of b_1 and b_2 on the right side of b_0 implies that the BST considers them as bigger in the ordering of nodes.

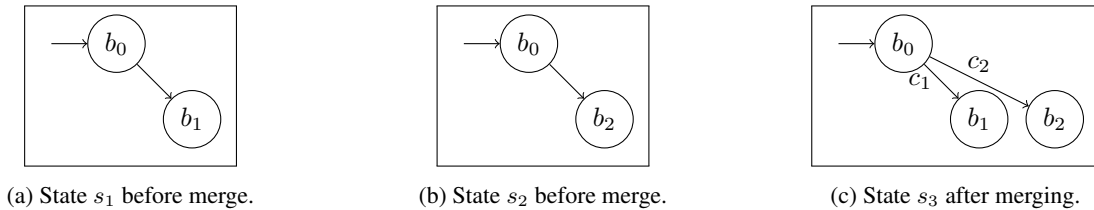


Figure 7.1: Visualisation of BST of the shadow memories.

After merging, the shadow memory of s_3 should correspond to the BST shown in Figure 7.1(c). Here, c_1 and c_2 denote the state conditions of s_1 and s_2 , respectively. The arrows in the figure represent internal pointers used by the data structure, rather than the publicly visible *next* pointers stored in the nodes themselves. Each internal pointer is annotated with the condition under which it is active. This contradicts the usual implementation of a BST, which has only one pointer for a child node that is bigger than the parent node. We’ll explain this in a moment.

Before discussing the implementation in more detail, we note an important property of c_1 and c_2 . As defined by ANGR, these conditions are mutually exclusive and exhaustive, i.e., exactly one of them holds in any model. Consequently, c_2 is equivalent to $\neg c_1$, and the state condition of the merged state s_3 is $c_1 \vee c_2$, which is equivalent to `True`.

We now consider a concrete lookup example to understand why there are now two child nodes that are bigger than the parent node. Suppose that in state s_1 the address *addr* points to a location l_1 within the buffer beginning at b_1 , and in state s_2 the same address points to a location l_2 within the buffer beginning at b_2 . After merging, the symbolic value of *addr* is

$$addr = ite(c_1, l_1, l_2).$$

Assume that the operation `ods.get_smaller_equal(addr)` is invoked. The expected result is

$$ite(c_1, b_1, b_2).$$

We propose an implementation as follows. To locate the buffer whose beginning address is smaller than or equal to *addr*, the BST traversal first compares *addr* with b_0 . Since $addr > b_0$ holds for all possible concretisations of *addr*, the traversal proceeds along the pointers to nodes with greater *begin* values.

Unlike a standard BST, there is now more than one such pointer, each annotated with a condition. For each outgoing pointer p with associated condition c_p , the solver checks whether the conjunction of the current state condition c_3 and c_p is satisfiable. If so, the traversal follows that pointer and records the corresponding condition as part of the path condition, denoted c_{path} .

In this example, the traversal follows both pointers, reaching node b_1 with path condition $c_{path} = c_1$ and node b_2 with path condition $c_{path} = c_2$. At node b_1 , the path condition implies that *addr* can only be concretised to l_1 , and the algorithm concludes that b_1 is the correct buffer beginning searched by the `get_smaller_equal` function under this condition. Analogously, at node b_2 , it concludes that b_2 is the correct buffer beginning. Using the accumulated path conditions, the final result $ite(c_1, b_1, b_2)$ is constructed.

This example illustrates only a simple case. In practice, the implementation would be considerably more complex. In particular, after multiple forks and merges, pointer conditions may no longer be mutually exclusive. It is possible to encounter models that satisfy multiple pointer conditions simultaneously, only a subset of them, or none at all. This complexity arises because pointer conditions may encode aggregated state conditions accumulated over several merges rather than simple state predicates.

Chapter 8

Implementation and Evaluation

Our implementation is open source and available at [7].

To demonstrate the practical feasibility of the proposed theoretical approach, we implemented a proof-of-concept. The implementation is based on the symbolic execution engine ANGR and makes use of data structures provided by the Python library SORTEDCONTAINERS [20].

We implemented the heap memory management vulnerability detection algorithm 5.2.1 using the concrete-chunk, address-based shadow memory implementation 6.2.2.1, as well as the heap memory usage vulnerability detection algorithm 5.2.2 using both the symbolic-chunk, address-based shadow memory implementation 6.2.2.1 and the symbolic-chunk, range-based shadow memory implementation 6.2.3.2. The latter algorithm was implemented with both shadow memory variants to facilitate future work, in particular to enable comprehensive performance evaluations and to compare their effectiveness in real-world applications.

8.1 Implementation

- **Memory Model and Environment Modeling:** We used the default memory model of ANGR together with its standard concretisation strategies. For memory management, we rely on the SIMPROCEDURES provided by the ANGR heap memory plugin SIMHEAPPTMALLOC, which implements heap behaviour following the *ptmalloc* design.
- **Detection Semantics:** The vulnerability detection algorithms 5.2.1 and 5.2.2 were implemented using ANGR's hooking functionality, which allows interception of SIMPROCEDURES. Memory access instrumentation was realised using the breakpoint mechanism provided by ANGR.
- **Shadow Memory Implementation:** The shadow memory was implemented as an ANGR plugin to ensure compatibility with state forking and merging. The detection algorithms implemented in hooks and breakpoints access this shadow memory to track poisoned and unpoisoned regions. Within this plugin, the implementations 6.2.2.1, 6.2.2.1 and 6.2.3.2 were realised.
- **Concretisation of Address Checks:** In addition to the shadow memory implementation, we provide an option to control shadow memory concretisation behaviour during address checks. Specifically, users can specify whether an address should be checked fully symbolically or according to the concretisations produced by the standard ANGR memory model.
- **Range-Based Shadow Memory:** As we did not identify suitable existing data structures for implementing range-based shadow memory, we developed a custom implementation using SORTEDLIST from the SORTEDCONTAINERS library. The SORTEDLIST stores the starting addresses of unpoisoned memory chunks. A dictionary maps each starting address to a node containing the corresponding size and a

pointer to the next node. Sorting is therefore performed solely on chunk start addresses, not on the whole nodes.

- **Optimisations:** We implemented some optimisations that modify the originally presented shadow memory algorithms. The most significant optimisation checks whether an address is symbolic and, if not, minimises solver invocations wherever possible.

8.2 Evaluation

We developed an extensive test suite consisting of 35 C++ programs and evaluated our implementation on all of them. The test cases are publicly available alongside the implementation to ensure reproducibility.

The evaluation primarily focuses on validating the correctness and robustness of the proposed detection semantics. To this end, the test cases were designed to systematically cover relevant execution scenarios and edge cases:

- Both fully concrete and fully symbolic execution scenarios were tested. For symbolic cases, we considered symbolic addresses whose concretisations span both contiguous and non-contiguous memory regions.
- Standard as well as edge cases of heap memory management functions were exercised.
- Boundary value analysis was performed for allocated heap buffers in the heap memory usage vulnerability detection algorithm.
- Scenarios involving multiple heap allocations were included to evaluate interactions between buffers and the behaviour of the shadow memory beyond isolated cases.
- We verified that invalid accesses to stack addresses are correctly ignored and do not trigger false alarms.
- Both detection algorithms were evaluated. In particular, the heap memory usage detection algorithm was tested with both shadow memory implementations and with address concretisation during checks both enabled and disabled.

We verified that the implementation does not produce unexpected false positives or false negatives across all test cases. For every detected vulnerability, we extracted and concretised the standard input from the corresponding ANGR state. Executing the program with this input successfully reproduced the vulnerability in all cases, providing strong empirical evidence that reported findings correspond to real, exploitable memory safety violations rather than artefacts of the analysis.

We did not conduct a systematic performance evaluation. Performance was not formulated as a primary objective of this thesis, and given the theoretical focus of this work, a comprehensive benchmark-based evaluation would exceed the intended scope of a Bachelor's thesis. And conducting such an evaluation would require first extending the implementation to incorporate the mentioned path-explosion mitigation techniques as they will be inevitable for the use on real-world software.

Nevertheless, ad-hoc performance observations were made during testing. Execution was particularly slow in fully symbolic scenarios, especially when symbolic pointers had non-contiguous concretisation ranges. Instrumentation and logging indicated that solver invocations were a dominant contributor to runtime, with individual queries taking approximately 0.2–0.4 seconds in our experimental setup (details on hardware, operating system, and configuration are provided alongside the implementation). While the runtime-complexity formulas presented in Chapter 6 provide useful asymptotic insight, practical performance was strongly influenced by constant factors, most notably solver interactions. Additional overhead was introduced by verbose logging, which frequently triggered solver queries. Finally, some performance degradation could not be fully attributed to a single cause and is assumed to result from a combination of symbolic execution overhead and hardware limitations.

Chapter 9

Discussion

We briefly restate the five primary objectives of this thesis:

- O_1 : Analyse symbolic memory models and identify the model that offers the best trade-off between correctness and resource usage.
- O_2 : Identify classes of memory access vulnerabilities detectable by the combination of symbolic execution and address sanitisation on binaries.
- O_3 : Design data structures and algorithms for implementing shadow memory with respect to space consumption, runtime overhead, and solver interaction.
- O_4 : Briefly discuss the applicability of established path-explosion mitigation techniques and outline their potential integration as future work.
- O_5 : Implement a proof-of-concept for heap memory vulnerability detection within the ANGR framework.

We provided an overview of symbolic memory models, particularly in binary-only contexts, and analysed the trade-off between precision and resource consumption. Based on theoretical analysis and existing data, we identified two memory models well-suited to our goals, and we focused on the default ANGR memory model. While we consider our analysis a valuable contribution, a substantially deeper analysis with larger empirical datasets would be required to draw more representative statistical conclusions. Nonetheless, our conclusions are applicable beyond the combination of symbolic execution and address sanitisation in a binary-only setting and may benefit other symbolic execution use cases. With that, we achieved objective O_1 .

Using the C language specifications of heap memory management functions, the System V AMD64 ABI for the stack, and reasonable assumptions about compiler behaviour, we successfully identified memory access vulnerabilities in a binary-only setting. By adhering closely to the C specification and the mentioned ABI, our approach remains applicable across a wide range of applications with minimal compiler-specific assumptions. We presented detection algorithms for these vulnerabilities, and thus, we achieved objective O_2 .

Nevertheless, we want to note that certain classes of vulnerabilities remain outside the scope of our approach. Binary-only analysis inherently lacks high-level object information that tools such as KLEE and Google ASAN can exploit. Moreover, because our approach relies on poisoned memory regions rather than base-pointer reasoning, some out-of-bounds accesses may go undetected when they fall into adjacent valid buffers. Addressing such cases would require additional static analysis. Despite these limitations, our method provides practical value by avoiding complex static analysis, not requiring source code, and enabling effective vulnerability detection of a large number of invalid memory access vulnerability types through a simple address sanitisation mechanism.

Furthermore, we examined multiple shadow memory implementations, including their algorithms, data structures, and complexity characteristics. For each detection algorithm, we identified the most suitable shadow memory implementation where sufficient data was available. This analysis is critical, as performance remains a key obstacle to industrial adoption of symbolic execution. With that, we achieved objective O_3 . Nonetheless, in

some cases, uncertainties persisted because little to no literature exists on stochastic data for symbolic pointers, making it difficult to choose a shadow memory implementation for a specific detection algorithm.

Path explosion mitigation techniques were reviewed and analysed with respect to their compatibility with our approach. We presented formal algorithms for state merging, providing a theoretical foundation for future work aimed at scalability. With that, we also achieved objective O_4 .

Finally, as formulated in objective O_5 we implemented a subset of the proposed detection algorithms using ANGR as a proof of concept. Extensive testing confirmed correctness, with no unexpected false positives or false negatives. However, performance limitations—primarily due to solver overhead—remain a challenge, highlighting the importance of implementing path-explosion mitigation techniques and optimising shadow memory implementation by reducing solver queries in future work.

Chapter 10

Conclusion

Invalid memory accesses are still one of the most severe and frequent sources of software vulnerabilities, as highlighted in the introduction by their widespread presence in vulnerability databases and real-world incidents. This thesis set out to provide the theoretical foundation for investigating whether and how symbolic execution and address sanitisation can be combined in a binary-only setting to address this problem in a principled and scalable manner.

We first analysed symbolic memory models with respect to the trade-off between correctness and resource usage and identified two memory models that are well suited for our purposes. We identified classes of invalid memory access vulnerabilities that can be detected in binaries using the proposed combination of techniques and provided the corresponding detection semantics. We designed and analysed several data structures and algorithms for implementing shadow memory, and identified the most suitable variant for each class of detectable vulnerabilities. To address the inherent scalability limitations of symbolic execution, we discussed the applicability of established path explosion mitigation techniques and their compatibility with our approach.

Finally, to demonstrate practical feasibility, we implemented a proof-of-concept for heap memory vulnerability detection within the ANGR framework. We evaluated its correctness with an extensive test suite, and all expected vulnerabilities were successfully detected and reproduced.

In summary, this thesis closes the gap identified in the introduction by providing a coherent theoretical and practical framework for combining symbolic execution and address sanitisation in a binary-only setting. While further work is required to improve scalability and performance, particularly through the implementation of path-explosion mitigation and through solver query number optimisations, the results presented here establish a solid foundation for future research and tool development aimed at detecting memory-safety vulnerabilities in real-world binaries.

Chapter 11

Future Work

11.1 Memory Models

The memory management model MEMSIGHT may be considered for future work because it supports fully symbolic memory accesses and thus doesn't introduce unsoundness and incompleteness. A deeper theoretical analysis of space and runtime of MEMSIGHT's memory model would be valuable. Also, implementing heap memory management SIMPROCEDURES for MEMSIGHT would make it much more attractive for use in symbolic execution. Additionally, investigating and implementing our proposed changes in 4.5 could be valuable.

11.2 Detection Semantics

For heap-related vulnerabilities, our binary-only address sanitisation approach is largely exhaustive. One remaining extension would be detecting accesses outside heap boundaries, which may require intercepting system calls such as `mmap`, `brk`, or `sbrk`. This would necessitate symbolic execution of memory management functions. One could also consider integrating functions such as `new` and `delete`, as well as standard library, third-party, or custom functions operating on unallocated heap memory, into our detection algorithms.

On the stack, additional assumptions about the compiler, operating system, architecture, and ABI could enable detection of further vulnerabilities, such as incorrect handling of callee-preserved registers. The advantage of using address sanitisation for such registers is that it can detect vulnerabilities earlier than shadow stack approaches, but it would be more complex to implement. Integrating reverse engineering techniques to recover object boundaries and identify unused padding regions could further improve precision. Explicit handling of exception mechanisms is another potential extension.

11.3 Data Structures and Algorithms for Shadow Memory

Future work could explore alternative data structures tailored to MEMSIGHT and analyze their runtime, space, and solver query complexity. Further optimisations to reduce solver invocations may significantly improve performance. Introducing controlled unsoundness or incompleteness — such as assuming contiguous concretisation ranges, as done in S2EDROID [32] — could also yield practical performance benefits. Additionally, formulating detection algorithms and corresponding shadow memory implementations for a fully sound and complete symbolic execution engine that does not introduce unsoundness or incompleteness like ANGR may be of academic interest.

11.4 Memory Models and Path Explosion

Further research could investigate how our algorithms can be adapted to support both precise and over-approximating state merging across all shadow memory implementations. Especially for the correctness-preserving state merging, it would be valuable to formalise a merging strategy for range-based shadow memory implementations.

11.5 Implementation

From an implementation perspective, extending the prototype to support all proposed detection algorithms would be a natural next step. In addition to the default ANGR memory model, MEMSIGHT could be evaluated in practice. However, the highest priority for future work is implementing multiple path-explosion mitigation techniques and rigorously evaluating performance on benchmarks and real-world programs.

11.6 Additional Directions

Future work could also explore relaxing or extending one of the three foundational pillars of this thesis: symbolic execution, address sanitisation, and binary-only analysis. For example, fuzzing techniques could be integrated alongside symbolic execution, or static analysis could complement address sanitisation. Finally, adapting the approach to source-level analysis — similar to KLEE — could further expand its applicability.

Chapter 12

Usage of AI

AI-assisted technologies were used selectively during the development of this thesis to support specific aspects of the writing, research, and implementation process. Their usage did not replace independent reasoning, analysis, or implementation. The following outlines how these tools were employed:

- **Text Transformation and Stylistic Improvement:** CHATGPT was used to improve the clarity, fluency, and professional tone of text passages without changing their technical content or meaning. Typical prompts were of the form:

Please rewrite the following text to sound more professional, without changing its technical content: <text>.

- **Research Support:** CHATGPT was used to obtain high-level overviews and summaries of research areas as a basis for orientation and literature exploration. These summaries did not replace independent research. All factual claims and technical details included in this thesis are supported by primary sources cited accordingly. Example prompts include:

Please provide an overview of state-of-the-art state merging strategies used in modern symbolic execution engines.

- **Implementation Clarification:** CHATGPT and LE CHAT were used to clarify the intended usage and behavior of specific components of the ANGR framework. This support was limited to understanding APIs and design choices and did not involve generating core implementation logic. Example prompts include:

Given a function name in ANGR, which analysis should be used to obtain the address of the first basic block: CFGFAST or CFGEMULATED?

- **Grammar and Spelling Checks:** GRAMMARLY was used to assist with grammar, spelling, and stylistic consistency throughout the thesis.

Chapter 13

Appendix

13.1 Pseudocode for Heap Memory Management Vulnerability Detection Semantics

Here we present the detailed version of the algorithm presented in 5.2.1 which detects heap memory management vulnerabilities.

Algorithm 12 Detecting heap memory management violations.

```
1: function INIT( )
2:   shadow.poison(1, memory.max_addr)
3:
4: function MALLOC_INSTRUM(size)
5:   ptr = memory.malloc(size)
6:   if ptr!=null then shadow.unpoison(ptr)
7:
8: function CALLOC_INSTRUM(nmemb, size)
9:   ptr = memory.calloc(nmemb, size)
10:  if ptr!=null then shadow.unpoison(ptr)
11:
12: function REALLOC_INSTRUM(argptr, size)
13:  if max(argptr) != null and satisfiable(shadow.is_poisoned(max(ptr))) then boom()
14:  retptr = realloc(argptr, size)
15:  if max(argptr) == null and retptr != null then shadow.unpoison(retptr)
16:  else if max(size) == 0 then shadow.poison(argptr)
17:  else if retptr != null then
18:    shadow.poison(argptr)
19:    shadow.unpoison(retptr)
20:
21: function FREE_INSTRUM(ptr)
22:  if satisfiable(shadow.is_poisoned(ptr)) then boom()
23:  memory.free(ptr)
24:  if max(ptr) != null then shadow.poison(max(ptr))
```

We don't repeat clarifications already made in 5.2.1. Since the instrumentation of `realloc` is non-trivial, we briefly explain its handling. Line 15 addresses the case where `argptr` is null, in which case `realloc`

behaves like `malloc`. Line 16 covers the case where `size` is zero, causing `realloc` to behave like `free`. Finally, Line 17 captures the standard `realloc` behavior, in which an existing allocation is resized.

13.2 Pseudocode for Heap Memory Usage Vulnerability Detection Semantics

Here we present the detailed version of the algorithm presented in 5.2.2 which detects heap memory usage vulnerabilities.

Algorithm 13 Detecting heap memory usage violations.

```
1: function INIT( )
2:   shadow.unpoison(1, memory.heap_begin - 1)
3:   shadow.poison(memory.heap_begin, memory.heap_end)
4:   shadow.unpoison(memory.heap_end + 1, memory.max_addr)
5:
6: function MALLOC_INSTRUM(size)
7:   ptr = memory.malloc(size)
8:   if ptr != null then
9:     shadow.unpoison(ptr:ptr+size)
10:    meta[ptr] = size
11:
12: function CALLOC_INSTRUM(nmemb, size)
13:   ptr = memory.calloc(nmemb, size)
14:   if ptr != null then
15:     shadow.unpoison(ptr:ptr+nmemb*size)
16:     meta[ptr] = nmemb*size
17:
18: function FREE_INSTRUM(ptr)
19:   memory.free(ptr)
20:   if max(ptr) != null then
21:     size = meta[ptr]
22:     shadow.poison(ptr:ptr+size)
23:
24: function REALLOC_INSTRUM(argptr, size)
25:   retptr = realloc(argptr, size)
26:   if max(argptr) == null and retptr != null then
27:     shadow.unpoison(retptr:retptr+size)
28:     meta[retptr] = size
29:   else if max(size) == 0 then
30:     size_of_argptr = meta[argptr]
31:     shadow.poison(argptr:argptr+size_of_argptr)
32:   else if retptr != null then
33:     size_of_argptr = meta[argptr]
34:     shadow.poison(argptr:argptr+size_of_argptr)
35:     shadow.unpoison(retptr:size)
36:     meta[retptr] = size
37:
38: function MEM_ACCESS_INSTRUM(addr, length)
39:   if satisfiable(shadow.contains_poisoned(addr:addr+length)) then boom()
40:   memory.mem_access(addr)
```

We do not repeat the clarifications already discussed in Section 5.2.1 or in 13.1. For more details, see the mentioned sections.

13.3 Pseudocode for Uninitialised Heap Memory Vulnerability Detection Semantics

Here we present the detailed version of the algorithm presented in 5.2.3 which detects uninitialised heap memory read vulnerabilities.

Algorithm 14 Detecting uninitialised heap memory reads.

```
1: function INIT( )
2:   shadow.unpoison(1, memory.heap_begin - 1)
3:   shadow.poison(memory.heap_begin, memory.heap_end)
4:   shadow.unpoison(memory.heap_end + 1, memory.max_addr)
5:
6: function MALLOC_INSTRUM(size)
7:   ptr = memory.malloc(size)
8:   if ptr!=null then
9:     shadow.poison(ptr:ptr+size)
10:    meta[ptr] = size
11:
12: function CALLOC_INSTRUM(nmemb, size)
13:   ptr = memory.calloc(nmemb, size)
14:   if ptr!=null then
15:     shadow.unpoison(ptr:ptr+nmemb*size)
16:     meta[ptr] = nmemb*size
17:
18: function FREE_INSTRUM(ptr)
19:   memory.free(ptr)
20:
21: function REALLOC_INSTRUM(argptr, size)
22:   retptr = realloc(argptr, size)
23:   if max(argptr) == null and retptr != null then
24:     shadow.poison(retptr:retptr+size)
25:     meta[retptr] = size
26:   else if max(size) == 0 then
27:     size_of_argptr = meta[argptr]
28:   else if retptr != null then
29:     size_of_argptr = meta[argptr]
30:     temp = shadow.get(argptr:argptr+size_of_argptr)
31:     if size_of_argptr > size then
32:       temp = temp[0:size-1]
33:     else if size_of_argptr < size then
34:       temp = temp + poisoned(size - size_of_argptr)
35:     shadow.set(retptr, temp)
36:     meta[retptr] = size
37:
38: function MEM_WRITE_INSTRUM(addr, length)
39:   shadow.unpoison(addr:addr+length)
40:   memory.mem_write(addr)
41:
42: function MEM_READ_INSTRUM(addr, length)
43:   if not satisfiable(shadow.contains_unpoisoned(addr:addr+length)) then boom()
44:   memory.mem_read(addr)
```

We do not repeat the clarifications already provided in Section 5.2.3 or in Section 13.1. Instead, we emphasize the following additional aspects:

- As already mentioned, for allocations performed via `malloc`, the entire allocated buffer is poisoned, as no initialised data is expected to reside in newly allocated memory. In contrast, allocations via `calloc`

result in the entire region being unpoisoned, since `calloc` initialises the memory to zero.

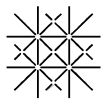
- During buffer reallocation with `realloc`, metadata describing poisoned regions within the original buffer are first stored in a temporary variable `temp` and subsequently restored at the new memory location.
- Any write operation unpoisons the corresponding memory region.

Bibliography

- [1] State plugins. https://docs.angr.io/en/latest/extending-angr/state_plugins.html, 21 January 2026.
- [2] Stackshield: A stack smashing technique protection tool for linux. 2000. <http://www.angelfire.com/sk/stackshield/>, accessed 13 January 2026.
- [3] BitBlaze binary analysis project. 2007. <http://bitblaze.cs.berkeley.edu>, accessed 12 January 2026.
- [4] INTERNATIONAL STANDARD ©ISO/IEC ISO/IEC 9899:2024 information technology — programming languages — c. 2024.
- [5] CWE VIEW: Weaknesses in the 2025 CWE top 25 most dangerous software weaknesses. 2025. <https://cwe.mitre.org/data/slices/1435.html>, accessed 09 January 2026.
- [6] GNU Gnulib. 2026. https://www.gnu.org/software/gnulib/manual/html_node/index.html#SEC_Contents1, accessed 13 January 2026.
- [7] Symbolic address sanitizer. 2026. <https://github.com/Emanuele-Tirendi/SymbolicAddressSanitizer/tree/main>, accessed 25 January 2026.
- [8] Thanassis Avgerinos, Sang Kil Cha, Brent Lim Tze Hao, and David Brumley. AEG: Automatic exploit generation. *Proceedings of the 36th International Conference on Software Engineering, ICSE 2014*, page 1083–1094, 2014.
- [9] Roberto Baldoni, Emilio Coppa, Daniele Cono D’Elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR), Volume 51, Issue 3*, pages 1–39, 2018.
- [10] Sara Baradaran, Mahdi Heidari, Ali Kamali, and Maryam Mouzarani. A unit-based symbolic execution method for detecting memory corruption vulnerabilities in executable codes. *International Journal of Information Security (2023) 22*, page 1277–1290, 2023.
- [11] Luca Borzacchiello, Emilio Coppa, Daniele Cono D’Elia, and Camil Demetrescu. Memory models in symbolic execution: key ideas and new thoughts. *Software Testing, Verification and Reliability Vol 29*, page e1722, 2019. <https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.1722>, accessed 12 January 2026.
- [12] David Brumley, Ivan Jager, Thanassis Avgerinos, , and Edward J Schwart. BAP: A binary analysis platform. *Proc. of the Conference on Computer Aided Verification*, 2011.
- [13] Muhammad Arif Butt, Zarafshan Ajmal, Zafar Iqbal Khan, Muhammad Idrees, and Yasir Javed. An in-depth survey of bypassing buffer overflow mitigation techniques. 2022.

- [14] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. *18th USENIX Conf. on Operating Systems Design and Implem, (OSDI '08)*, page 209–224, 2008.
- [15] Sang Kil Cha, Thanassis Avgerinos, Alexandre Rebert, and David Brumley. Unleashing MAYHEM on binary code. *2012 IEEE Symposium on Security and Privacy*, pages 380–394, 2012.
- [16] DARPA. CGC: Cyber grand challenge. 2016.
- [17] Robin David, Sebastien Bardin, Thanh Dinh Ta, Josselin Feist, Laurent Mounier, and Marie-Laure Potet. BINSEC/SE: A dynamic symbolic execution toolkit for binary-level analysis. 2016. <https://doi.org/10.1109/SANER.2016.43>.
- [18] Leila Delshadtehrani, Schuyler Eldridge, Student Member Sadullah Canakci, Manuel Egele, and Ajay Joshi. Nile: A programmable monitoring coprocessor. *IEEE COMPUTER ARCHITECTURE LETTERS, VOL. 17*, 2018.
- [19] Patrick Horgan. Linux x86 program start up or - how the heck do we get to main()? <http://dbp-consulting.com/tutorials/debugging/linuxProgramStartup.html>, accessed 13 December 2025.
- [20] Grant Jenks. Sorted containers. <https://grantjenks.com/docs/sortedcontainers/>, accessed 22 January 2026.
- [21] Sean Michael Kerner. Heartbleed SSL flaw’s true cost will take time to tally. 2014. <https://www.eweek.com/security/heartbleed-ssl-flaw-s-true-cost-will-take-time-to-tally/>, accessed 09 January 2026.
- [22] H.J. Lu, Michael Matz, Milind Girkar, Jan Hubicka, Andreas Jaeger, and Mark Mitchell. System V application binary interface, AMD64 architecture processor supplement (with LP64 and ILP32 programming models) version 1.0. 2025.
- [23] Lorenzo Martignoni, Stephen McCamant, Pongsin Poosankam, Dawn Song, and Petros Maniatis. Path-exploration lifting: Hi-fi tests for lo-fi emulators. *Proceedings of the Seventeenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVII)*, page 337–348, 2012.
- [24] Saahil Ognawala, Martín Ochoa, Alexander Pretschner, and Tobias Limmer. MACKE: Compositional analysis of low-level vulnerabilities with symbolic execution. *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 780–785, 2016.
- [25] H. Ozdoganoglu, T.N. Vijaykumar, C.E. Brodley, B.A. Kuperman, and A. Jalote. SmashGuard: A hardware solution to prevent attacks on the function return address. *IEEE Transactions on Computersc, Vol 55*, pages 1271–1285, 2006.
- [26] Ahmed Rezine. Symbolic execution. 2022. <https://www.ida.liu.se/~TDDD04/lectures/slides/2022/mcse.pdf>, accessed 13 January 2026.
- [27] Henry Gordon Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society 74*, pages 358–366, 1953.
- [28] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. AddressSanitizer: A fast address sanity checker. 2012.
- [29] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. SOK: (state of) the art of war: offensive techniques in binary analysis. *Proceedings of the 2016 IEEE Symposium on Security and Privacy (SP '16)*, page 138–157, 2016.

- [30] Aditya Thakur, Junghee Lim, Akash Lal, Amanda Burton, Evan Driscoll, Matt Elder, Tycho Andersen, and Thomas Reps. Directed proof generation for machine code. *Computer Aided Verification*, page 288–305, 2010.
- [31] Germán Vidal. Closed symbolic execution for verifying program termination. *2012 IEEE 12th International Working Conference on Source Code Analysis and Manipulation*, pages 34–43, 2012.
- [32] Bin Zhang, Bo Wu, Chao Feng, and Chaojing Tang. Memory corruption vulnerabilities detection for android binary software. *2015 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC)*, pages 1–5, 2015.



Declaration on Scientific Integrity

(including a Declaration on Plagiarism and Fraud)

Translation from German original

Title of Thesis: Detecting Memory Safety Violations in Binary-Only Programs Using Symbolic Execution and Address Sanitisation

Name Assessor: Dr. Marco Vogt

Name Student: Emanuele Tirendi

Matriculation No.: 21-052-089

I attest with my signature that I have written this work independently and without outside help. I also attest that the information concerning the sources used in this work is true and complete in every respect. All sources that have been quoted or paraphrased have been marked accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using the appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or expulsion from the study program.

Place, Date: 28.01.2026

Student:

Emanuele Tirendi

Will this work, or parts of it, be published?

☐

No

☒

Yes. With my signature I confirm that I agree to a publication of the work (print/digital) in the library, on the research database of the University of Basel and/or on the document server of the department. Likewise, I agree to the bibliographic reference in the catalog SLSP (Swiss Library Service Platform). (cross out as applicable)

Publication as of: 29.01.2026

Place, Date: 28.01.2026

Student:

Emanuele Tirendi

Place, Date:

Assessor:

Please enclose a completed and signed copy of this declaration in your Bachelor's or Master's thesis.